

```

// In production, we register a service worker to serve assets from local cache.
// This lets the app load faster on subsequent visits in production, and gives
// it offline capabilities. However, it also means that developers (and users)
// will only see deployed updates on the "N+1" visit to a page, since previously
// cached resources are updated in the background.
// To learn more about the benefits of this model, read https://goo.gl/KwvDNy.
// This link also includes instructions on opting out of this behavior.

const isLocalhost = Boolean(
  window.location.hostname === 'localhost' ||
    // [::1] is the IPv6 localhost address.
    window.location.hostname === '[::1]' ||
    // 127.0.0.1/8 is considered localhost for IPv4.
    window.location.hostname.match(
      /^127(?:\.(?:25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)?)?$/
    )
);

export default function register() {
  if (process.env.NODE_ENV === 'production' && 'serviceWorker' in navigator) {
    // The URL constructor is available in all browsers that support SW.
    const publicUrl = new URL(process.env.PUBLIC_URL, window.location);
    if (publicUrl.origin !== window.location.origin) {
      // Our service worker won't work if PUBLIC_URL is on a different origin
      // from what our page is served on. This might happen if a CDN is used to
      // serve assets; see https://github.com/facebookincubator/create-react-app/issues/2374
      return;
    }
    window.addEventListener('load', () => {
      const swUrl = `${process.env.PUBLIC_URL}/service-worker.js`;
      if (!isLocalhost) {
        // Is not local host. Just register service worker
        registerValidSW(swUrl);
      } else {
        // This is running on localhost. Lets check if a service worker still exists or not.
        checkValidServiceWorker(swUrl);
      }
    });
  }
}

function registerValidSW(swUrl) {
  navigator.serviceWorker
    .register(swUrl)
    .then(registration => {
      registration.onupdatefound = () => {
        const installingWorker = registration.installing;
        installingWorker.onstatechange = () => {
          if (installingWorker.state === 'installed') {
            if (navigator.serviceWorker.controller) {
              // At this point, the old content will have been purged and
              // the fresh content will have been added to the cache.
              // It's the perfect time to display a "New content is
              // available; please refresh." message in your web app.
              console.log('New content is available; please refresh.');
```

```

function checkValidServiceWorker(swUrl) {
  // Check if the service worker can be found. If it can't reload the page.
  fetch(swUrl)
    .then(response => {
      // Ensure service worker exists, and that we really are getting a JS file.
      if (
        response.status === 404 ||
        response.headers.get('content-type').indexOf('javascript') === -1
      ) {
        // No service worker found. Probably a different app. Reload the page.
        navigator.serviceWorker.ready.then(registration => {
          registration.unregister().then(() => {
            window.location.reload();
          });
        });
      } else {
        // Service worker found. Proceed as normal.
        registerValidSW(swUrl);
      }
    })
    .catch(() => {
      console.log(
        'No internet connection found. App is running in offline mode.'
      );
    });
}

export function unregister() {
  if ('serviceWorker' in navigator) {
    navigator.serviceWorker.ready.then(registration => {
      registration.unregister();
    });
  }
}

import NetworkUtils from './NetworkUtils';

const AccountApi = {
  /**
   * 帐号密码登录
   * @param username 帐号
   * @param password 密码
   */
  login: function (username = '', password = '') {
    return NetworkUtils.post('/login', {
      username: username,
      password: password
    });
  },
  /**
   * 退出登录
   */
  logout: function () {
    return NetworkUtils.del('/logout');
  },
  /**
   * 获取登录用户信息
   */
  getSessionUserInfo: function (params) {
    return NetworkUtils.get('/user', params);
  },
  /**
   * Login By Token
   */
  loginByToken: function (params) {
    return NetworkUtils.post('/loginByToken', params);
  }
}

```

```

    },
    /**
     * 获取用户Score信息
     */
    getFDTScore: function (userId) {
        return NetworkUtils.get(`/users/${userId}/features`);
    },
    loginFromPartner: function (params) {
        return NetworkUtils.post(`/partner/login`, params);
    },
    loginByTrialCode: function (params) {
        return NetworkUtils.post(`/public/trial/signin`, params);
    },
    giveMoney: function (params) {
        return NetworkUtils.post(`/user/giveMoney`, params);
    },
};

export default AccountApi;

import NetworkUtils from './NetworkUtils';

const BetGameApi = {
    getBetGameHistory: function ({page=1, pageSize=20}) {
        return NetworkUtils.get(`/betGames/history`, {
            page, pageSize
        });
    },
    getBetGameResult: function ({betGameId}) {
        return NetworkUtils.get(`/betGames/${betGameId}/result`, {
        });
    },
    getBetGamePuzzle: function (puzzleId) {
        return NetworkUtils.get(`/puzzles/${puzzleId}`, {
        });
    }
};

export default BetGameApi;

import NetworkUtils from './NetworkUtils';

const CommonApi = {
    getWxSDKSignature: function (url) {
        return NetworkUtils.post(`/public/wx/jsSdkConfig`, {
            url: url
        });
    }
};

export default CommonApi;

import NetworkUtils from './NetworkUtils';

const ContestApi = {
    getZones: function () {
        return NetworkUtils.get(`/zones`, {
        });
    },
    getPlayingMatches: function () {
        return NetworkUtils.get(`/contests/playing`, {
        });
    }
};

```

```

    });
  },

  getActiveContest: function (contestId) {
    return NetworkUtils.get(`/contests/${contestId}/active`, {
    });
  },

  getContestResult: function (contestId) {
    return NetworkUtils.get(`/contests/${contestId}/result`, {
    });
  },

  getContestReplayData: function (contestId) {
    return NetworkUtils.get(`/contests/${contestId}`, {
    });
  },

  getContestHistoryList: function (page = 1, pageSize = 20) {
    return NetworkUtils.get(`/contests/history`, {
      page,
      pageSize
    });
  }
};
export default ContestApi;

import NetworkUtils from './NetworkUtils';

const IndicatorPracticeApi = {
  getNewIndicatorPractice: function ({indicatorId}) {
    return NetworkUtils.get(`/training/indicator_practice`, {
      indicatorId: indicatorId
    });
  },

  getTrainingStats: function () {
    return NetworkUtils.get(`/training/stats`);
  },

  saveIndicatorPractice: function ({practiceId, diffResult, recogResult, tradingResult}) {
    return NetworkUtils.post(`/training/indicator_practices/${practiceId}`, {
      diffResult, recogResult, tradingResult
    });
  }
};
export default IndicatorPracticeApi;

const JSONP = {
  // 获取当前时间戳
  now: function() {
    return (new Date()).getTime();
  },
  // 获取随机数
  rand: function() {
    return Math.random().toString().substr(2);
  },
  removeElem: function(elem) {
    let parent = elem.parentNode;
    if(parent && parent.nodeType !== 11) {
      parent.removeChild(elem);
    }
  },
};

```

```

getJSON: function(url, data, func) {
  // 函数名称
  let name;
  // 检测callback的函数名是否已经定义
  let match = /cb=(\w+)/.exec(url);
  if (match && match[1]) {
    name = match[1];
  } else {
    // 如果未定义函数名的话随机成一个函数名
    // 随机生成的函数名通过时间戳拼16位随机数的方式，重名的概率基本为0
    // 如:jsonp_1355750852040_8260732076596469
    name = "jsonp_" + this.now() + '_' + this.rand();
    // 把callback中的?替换成函数名
    url = url.replace("cb=?", "cb="+name);
    // 处理?被encode的情况
    url = url.replace("cb=%3F", "cb="+name);
  }
  // 创建一个script元素
  let script = document.createElement("script");
  script.type = "text/javascript";
  // 设置要远程的url
  script.src = url;
  // 设置id, 为了后面可以删除这个元素
  script.id = "id_" + name;
  // 把传进来的函数重新组装, 并把它设置为全局函数, 远程就是调用这个函数
  window[name] = function(json) {
    // 执行这个函数后, 要销毁这个函数
    window[name] = undefined;
    // 获取这个script的元素
    let elem = document.getElementById("id_" + name);
    // 删除head里面插入的script, 这三步都是为了不影响污染整个DOM啊
    JSONP.removeElem(elem);
    // 执行传入的的函数
    func(json);
  };
  // 在head里面插入script元素
  let head = document.getElementsByTagName("head");
  if (head && head[0]) {
    head[0].appendChild(script);
  }
}
};

export default JSONP;

import ifetch from 'isomorphic-fetch';
import queryString from 'query-string';
import SessionService from '../service/SessionService';

//const host = 'dev.hkfdt.cn';
const host = window.location.host;
const apiBaseUrl = '/api/v1';
const fetch = (url, options) => {
  const opt = options || {};
  const apiUrl = `${window.location.protocol}://${host}${apiBaseUrl}${url}`;
  return ifetch(apiUrl, {
    credentials: 'include',
    ...opt,
  }).then((response) => {
    if (response.status >= 200 && response.status < 300) {
      return response;
    }
  })
}

```

```

    if(response.status === 401){
      console.log('login and retry')
      return SessionService.restoreSessionIfTokenIsExit().then((data)=>{
        return ifetch(apiUrl, {
          credentials: 'include',
          ...opt,
        });
      });
    }
    const error = new Error(response.statusText);
    error.response = response;
    throw error;
  }).then(response => {
    if (parseInt(response.headers.get("content-length"), 10) === 0) {
      return null;
    } else {
      return response.json();
    }
  });
};

function toGetMethod(methodName) {
  return (url, params, options) => fetch((url + '?' + queryString.stringify(params)),
    { ...options, method: methodName });
}

function toMethod(methodName) {
  return (url, params, options) => fetch(url,
    { ...options, method: methodName, headers: {'Content-Type': 'application/x-www-form-urlencoded;charset=utf-8' }, body: queryString.stringify(params) });
}

const get = toGetMethod('get');
const del = toGetMethod('delete');
const post = toMethod('post');
const put = toMethod('put');
const postJson = (url, params, options) => fetch(url,
  { ...options, method: 'post', headers: {'Content-Type': 'application/json;charset=utf-8' }, body:
  JSON.stringify(params)});

const putFile = (url, formData) => {
  const apiUrl = `${window.location.protocol}//${host}${apiBaseUrl}${url}`;
  const req = new Request(apiUrl, {
    method: "PUT",
    body: formData,
    credentials: 'include',
  });
  return ifetch(req);
}

function handleError(data, errorMsg, callback) {
  if(data.response === undefined) {
    callback(errorMsg);
  } else {
    data.response.json().then(({code, error_msg})=>{
      callback(error_msg?`${errorMsg}, ${error_msg}`:errorMsg);
    });
  }
}

export default {
  fetch,
  get,
  post,
  put,
  del,
  postJson,
  putFile,
  handleError,
};

```

```
import NetworkUtils from './NetworkUtils';

const PatternPracticeApi = {
  getNewPatternPractice: function ({patternId}) {
    return NetworkUtils.get('/training/pattern_practice', {
      patternId: patternId
    });
  },
  getTrainingStats: function () {
    return NetworkUtils.get('/training/stats');
  },
  savePatternPractice: function ({practiceId, diffResult, recogResult, tradingResult, guessResult}) {
    return NetworkUtils.post(`/training/pattern_practices/${practiceId}`, {
      diffResult, recogResult, tradingResult, guessResult
    });
  }
};
export default PatternPracticeApi;
```

```
import NetworkUtils from './NetworkUtils';

const RankingApi = {
  getRichList: ({offset=0, limit=20}={})=>{
    return NetworkUtils.get('/public/users', {
      offset,
      limit,
      sortBy: 'money desc'
    });
  },
  getScoreList: ({offset=0, limit=20}={})=>{
    return NetworkUtils.get('/public/users', {
      offset,
      limit,
      sortBy: 'fdtscore desc'
    });
  },
};
export default RankingApi;
```

```
import JSONP from './JSONP';
import NetworkUtils from './NetworkUtils';

const StockApi = {
  /**
   * 股票代码检索
   */
  querySuggestion: function (keyword) {
    return new Promise((resolve, reject) => {
      try {
        let type = "stock";
        let timestamp = JSONP.now();
        //let url =
        `${window.location.protocol}//${window.location.host}/vendor/ifeng/api/suggest_v2.php?t=${type}&q=${keyword}&_=${timestamp}&cb=jsonp_${timestamp}(suggest_json)`;
        let url =
        `${window.location.protocol}//${window.location.host}/api/v1/stocks?t=${type}&q=${keyword}&_=${timestamp}&cb=jsonp_${timestamp}(suggest_json)`;
        JSONP.getJSON(url, null, function(json) {
          resolve(json);
        });
      } catch (e) {
        reject(e);
      }
    });
  },
};
```

```

/**
 * 查询相似K线
 */
querySimilarKlines: function (stockCode = "") {
  return NetworkUtils.get(`/similarKline/${stockCode}`, {});
},
/**
 * 查询我的自选股列表
 */
queryFavorites: function () {
  return NetworkUtils.get(`/user/selectedStocks`, {});
},
/**
 * 加入我的自选股
 */
addToFavorites: function (stockCode, stockName) {
  if (stockCode) {
    return NetworkUtils.post(`/user/selectedStocks/${stockCode}`, {
      symbol: stockCode,
      name: stockName
    });
  }
},
/**
 * 从我的自选股删除
 */
removeFromFavorites: function (stockCode) {
  if (stockCode) {
    return NetworkUtils.del(`/user/selectedStocks/${stockCode}`, {});
  }
},
};

export default StockApi;

import NetworkUtils from './NetworkUtils';

const TrainApi = {
  getNewTrain: function (patternId = '') {
    return NetworkUtils.get(`/trains/new`, {
      patternId: patternId
    });
  },
  getTrainingCount: function ({patternId}) {
    return NetworkUtils.get(`/trains/count`, {patternId});
  },
  getTrainingResult: function (trainingId) {
    return NetworkUtils.get(`/trains/${trainingId}/result`, {
    });
  },
  // ===== 获取练习统计数据 =====
  getPracticeStats: function () {
    return NetworkUtils.get(`/training/stats`);
  },
  // ===== 新形态训练 =====
  getPatternsMap: function () {
    return NetworkUtils.get(`/patterns/map`);
  },
  getPatternDetails: function (patternId) {
    return NetworkUtils.get(`/patterns/${patternId}`);
  },
  // ===== 指标买卖信号训练 =====
  getIndicatorsMap: function () {
    return NetworkUtils.get(`/indicators/map`);
  },
};

```



```

    getIndicatorDetails: function (indicatorId) {
      return NetworkUtils.get(`/indicators/${indicatorId}`);
    },
  };
};
export default TrainApi;

import React, { Component } from 'react';
import klineChartOption from './KlineChartOption';
import translateFromOldFormatData from './KlineDataAdapter';
import echarts from '../echarts';
import renderer from './renderer';
import './KlineChart.scss';
import splitHisData from './splitHisData';

class KlineChart extends Component {
  state = {
    seriesMax: 0,
    seriesMid: 0,
    seriesMin: 0
  }
  resize = () => {
    this.chart.resize();
  }
  componentDidMount() {
    const { historyData } = this.props;
    const oldFormatData = splitHisData(historyData);
    const data = translateFromOldFormatData(oldFormatData);
    this.setState({
      seriesMax: data.seriesMax,
      seriesMid: data.seriesMid,
      seriesMin: data.seriesMin
    });
    // draw kline chart
    const option = klineChartOption(data);
    this.chart = echarts.init(this.chartBoard, null, {renderer});
    this.chart.setOption(option);
  }

  componentWillUnmount() {
    echarts.dispose(this.chartBoard);
  }
  render() {
    return (
      <div className="fdt-kline-chart-wrapper">
        <div className="kline-chart" ref={node => (this.chartBoard = node)} />
        <span className="price-label highest">{this.state.seriesMax}</span>
        <span className="price-label mid">{this.state.seriesMid}</span>
        <span className="price-label lowest">{this.state.seriesMin}</span>
      </div>
    );
  }
}
export default KlineChart;

import CommonUtil from '../utils/CommonUtil';

export default function translateFromOldFormatData(oldFormatData) {
  // category (日期)
  let categoryData = [];
  for (let i = 0; i < oldFormatData.category.length; i++) {
    categoryData.push(CommonUtil.getFormatDateString(oldFormatData.category[i]));
  }
  // volume 交易量
  let volumes = [];
  for (let i = 0; i < oldFormatData.volumeUp.length; i++) {

```

```

    if (oldFormatData.volumeUp[i] !== 0) {
      volumes.push([i, oldFormatData.volumeUp[i], 1]);
    } else {
      volumes.push([i, oldFormatData.volumeDown[i], -1]);
    }
  }
  // MACD diff
  let diff = [];
  for (let i = 0; i < oldFormatData.diff.length; i++) {
    diff.push(oldFormatData.diff[i]);
  }
  // MACD dea
  let dea = [];
  for (let i = 0; i < oldFormatData.dea.length; i++) {
    dea.push(oldFormatData.dea[i]);
  }
  return {
    categoryData: categoryData,
    values: oldFormatData.series,
    volumes: volumes,
    diff: diff,
    dea: dea,
    ma5: oldFormatData.ma5,
    ma10: oldFormatData.ma10,
    ma20: oldFormatData.ma20,
    macdUp: oldFormatData.macdUp,
    macdDown: oldFormatData.macdDown,
    macdTipMax: oldFormatData.macdTipMax,
    seriesMax: oldFormatData.seriesMax,
    seriesMid: oldFormatData.seriesMid,
    seriesMin: oldFormatData.seriesMin
  };
}

```

// 数据处理方法 ---用于历史行情

import Decimal from 'decimal.js';

/**

* [splitHisData 拆分历史行情数据]

* @param {array} rawData [数据来源]

* @return {obj}

* category [类目轴数据]

* series [行情数据]

* volumeUp [成交量（涨）]

* volumeDown [成交量（跌）]

* ma5 [5日均线]

* ma10 [10日均线]

* ma20 [20日均线]

* seriesMax, [行情最大值]

* seriesMin, [行情最小值]

* volumeMax [成交量最大值]

*/

export default function splitHisData(rawData) {

const { min, max } = Math;

const {timestamp, oclh, volume, ma5, ma10, ma20, dea, diff, macd} = rawData;

const length = rawData.volume.length;

const lastIndex = length - 1;

const category = timestamp;

const series = rawData.oclh;

const volumeUp = [];

const volumeDown = [];

const macdUp = [];

const macdDown = [];

const macdName = 'MACD(12,26,9)';

const lastMa5 = ma5[lastIndex];

const lastMa10 = ma10[lastIndex];

```

const lastMa20 = ma20[lastIndex];
const lastVolume = volume[lastIndex];
let seriesMax = oclh[0][3];
let seriesMin = oclh[lastIndex][2];

volume.forEach((t, i) => {
  if (macd[i] > 0) {
    macdUp.push(macd[i]);
    macdDown.push(0);
  } else {
    macdUp.push(0);
    macdDown.push(macd[i]);
  }

  if(oclh[i][1]!==null){
    if (oclh[i][0] > oclh[i][1]) {
      volumeDown.push(volume[i]);
      volumeUp.push(0);
    } else {
      volumeUp.push(volume[i]);
      volumeDown.push(0);
    }
    if (seriesMax < oclh[i][3]) {
      seriesMax = oclh[i][3];
    }
    if (seriesMin > oclh[i][2]) {
      seriesMin = oclh[i][2];
    }
  }else{
    volumeDown.push(null);
    volumeUp.push(null);
  }
});

const seriesMid = Decimal.div(Decimal.add(seriesMin, seriesMax), 2).toNumber();
const macdMax = Decimal.abs(max.apply({}, macdUp));
const macdMin = Decimal.abs(min.apply({}, macdDown));
const diffMax = Decimal.abs(max.apply({}, diff));
const diffMin = Decimal.abs(min.apply({}, diff));
const deaMax = Decimal.abs(max.apply({}, dea));
const deaMin = Decimal.abs(min.apply({}, dea));
const macdTipMax = max.apply({}, [macdMax, macdMin, diffMax, diffMin, deaMax, deaMin]);
const volumeMax = max.apply({}, [max.apply({}, volumeUp), max.apply({}, volumeDown)]);

return {
  category,
  series,
  volumeUp,
  volumeDown,
  ma5,
  ma10,
  ma20,
  macdUp,
  macdDown,
  macdName,
  dea,
  diff,
  seriesMax,
  seriesMin,
  seriesMid,
  volumeMax,
  lastMa5,
  lastMa10,
  lastMa20,
  lastVolume,
  macdTipMax,
};
}

```

```
// 以前的色值, 备份用
// const colors = {
//   background: 'transparent', // 背景色
//   baseline: 'rgba(37,39,45,1.0)', // 分割线
//   priceline: 'rgba(140,158,173,1.0)', // 分时图走势线
//   basetext: '#A3AFB8',
//   blue: '#296c9e',
//   yellow: '#e6a648',
//   buy: '#C73827',
//   sell: '#4d9e75',
//   ma5: '#296c9e',
//   ma10: '#e6a648',
//   ma20: '#a473b2',
//   ma: 'rgba(230,166,72,0.5)',
//   closeprice: '#296c9e',
//   volume: '#cc7832',
//   diff: '#e6a648',
//   dea: '#296c9e',
// };
```

```
// 浅主题色
// const lightTheme = {
//   backgroundColor: 'transparent',
//   upColor: '#C73827',
//   downColor: '#4d9e75',
//   splitLineColor: 'rgba(0,0,0,0.1)',
//   labelColor: 'rgba(0,0,0,0.8)',
//   ma5: '#296c9e',
//   ma10: '#e6a648',
//   ma20: '#a473b2',
//   diff: '#e6a648',
//   dea: '#296c9e',
//   timingPriceLineColor: 'rgba(140,158,173,1.0)',
//   timingMaPriceLineColor: 'rgba(230,166,72,0.5)',
// }
```

```
// 深主题色
const darkTheme = {
  backgroundColor: 'transparent',
  upColor: 'rgba(255,57,0,1.0)',
  downColor: 'rgba(53,158,46,1.0)',
  splitLineColor: 'rgba(255,255,255,0.1)',
  labelColor: 'rgba(255,255,255,0.4)',
  ma5: '#296c9e',
  ma10: '#e6a648',
  ma20: '#a473b2',
  diff: 'rgba(230,166,72,0.5)',
  dea: 'rgba(255,57,0,1.0)',
  timingPriceLineColor: '#47BBFF',
  timingMaPriceLineColor: 'rgba(230,166,72,0.5)',
  yieldCurveColors: [ // 收益曲线选用颜色群
    "#d95850",
    "#73a373",
    "#73b9bc",
    "#7289ab",
    "#eb8146",
    "#d7504b",
    "#893448",
    "#27727b",
    "#ffb248",
    "#f2d643",
    "#dd6b66",
    "#759aa0",
    "#e69d87",
    "#8dc1a9",
    "#ea7e53",
    "#f49f42",
```

```

        "#b5c334",
        "#fe8463",
        "#9bca63",
        "#f3a43b",
        "#60c0dd",
        "#d7504b",
    ],
}

export default darkTheme;

import echarts from 'echarts/lib/echarts';
import 'zrender/lib/svg/svg';
import 'echarts/lib/chart/candlestick';
import 'echarts/lib/chart/line';
import 'echarts/lib/chart/bar';
import 'echarts/lib/component/grid';
import 'echarts/lib/component/axis';
import 'echarts/lib/component/markArea';
import 'echarts/lib/component/tooltip';
import 'echarts/lib/chart/radar';
import 'echarts/lib/component/visualMap';
import 'echarts/lib/component/graphic';
import 'echarts/lib/component/markPoint';
import 'echarts/lib/component/markLine';
import 'echarts/lib/component/dataZoom';
import 'echarts/lib/component/brush';
import 'echarts/lib/chart/custom';

export default echarts;

const renderer = (function(){
    if(navigator.userAgent.indexOf("iPhone") > 0) {
        return 'canvas';
    } else {
        return 'svg';
    }
})();

export default renderer;

export default {
    // 交易动作
    TRADE_ACTIONS: {
        BUY: 'B', // 买入
        SELL: 'S', // 卖出
        CLOSE: 'C' // 平仓
    },
    // 交易动作
    TRADE_ACTIONS_TEXT: {
        'B': '买', // 买入
        'S': '卖', // 卖出
        'C': '平' // 平仓
    },
    // 竞赛类型
    CONTEST_TYPE: {
        RANDOM_PUBLIC: 0, // 随机配对
        PRIVATE_ROOM: 1 // 私人房间
    },
    // 竞赛状态
    CONTEST_STATUS: {
        READY: 'READY',

```

```

    START: 'START',
    END: 'END',
    CANCEL: 'CANCEL'
  },
  MARKET_STATUS: {
    NOT_OPEN: 'NOT_OPEN',
    OPEN: 'OPEN',
    CLOSE: 'CLOSE'
  },
  // 竞猜游戏状态
  BET_GAME_STATUS: {
    READY: 'READY',
    LOCKED: 'LOCKED',
    OPEN: 'OPEN',
    STOP: 'STOP'
  },
  // 竞赛排名标识
  RANKING_INDICATOR: {
    BY_SCORE: 0,
    BY_ROI: 1
  },
  DEFAULT_AVATAR: 'https://www.timeshighereducation.com/sites/default/files/byline_photos/default-
avatar.png',
}

```

```

import React, { Component } from 'react';
import PropTypes from 'prop-types';
import './ActionButtonPanel.scss';

```

```

class ActionButtonPanelline extends Component {
  render() {
    return (
      <div className="line" />
    );
  }
}

```

```

class ActionButtonPanelButton extends Component {
  render() {
    return (
      <div className="button-wrapper"
        onClick={() => (!this.props.disabled && this.props.onClick && this.props.onClick())}
        style={{flexGrow:this.props.weight, opacity:(this.props.disabled?0.3:1.0)}}>
        <div className={`button ${this.props.type}`}>
          {this.props.children}
        </div>
      </div>
    );
  }
}

```

```

ActionButtonPanelButton.types = {
  NORMAL: "normal",
  HOLLOW: "hollow",
  BUY: "buy",
  SELL: "sell",
  CLOSE: "close",
  GRAY: "gray"
}

```

```

ActionButtonPanelButton.propTypes = {
  onClick: PropTypes.func,
  type: PropTypes.string,
  weight: PropTypes.number,
  disabled: PropTypes.bool
}

```

```

ActionButtonPanelButton.defaultProps = {
  onClick: null,
  type: ActionButtonPanelButton.types.NORMAL,
  weight: 1,
  disabled: false
};

class ActionButtonPanel extends Component {
  render() {
    return (
      <div className={`action-button-panel-wrapper ${this.props.className?this.props.className:''}`}
        style={this.props.style}>
        {this.props.children}
      </div>
    );
  }
}

ActionButtonPanel.Button = ActionButtonPanelButton;
ActionButtonPanel.Line = ActionButtonPanelLine;

export default ActionButtonPanel;

import React from 'react';
import './BackHomeOverlapBar.scss';

class BackHomeOverlapBar extends React.Component {

  state = {
    positionClass: 'offsetPosition'
  }

  show = () => {
    this.setState({
      positionClass: 'normalPosition'
    });
  }

  hide = () => {
    this.setState({
      positionClass: 'offsetPosition'
    });
  }

  render() {
    return (
      <div className={`backHomeOverlapBar ${this.state.positionClass}`} onClick={this.props.onClick}>
        <div className="overlapNavigatorPanelIcon">
          <div className="iconWhiteSubFrame"></div>
          <div className="iconWhiteSubFrame"></div>
          <div className="iconWhiteSubFrame"></div>
          <div className="iconWhiteSubFrame"></div>
        </div>
        <div className="backHomeOverlapBarText">首页</div>
      </div>
    );
  }
}

export default BackHomeOverlapBar;

import React from 'react';
import ReactDOM from 'react-dom';

```

```

import './BounceCountDown.scss';

class BounceCountDown extends React.Component {

  state = {
    show: false,
    countDownSeconds: 10,
    animatedStyleClasses: ""
  }

  componentDidMount() {
    this.setState({
      countDownSeconds: this.props.seconds,
      show: true,
    }, () => {
      this.watchAnimationCycle();
      this.beginCountDown();
    });
  }

  watchAnimationCycle = () => {
    let countDownDiv = document.getElementById("count-down-component");
    countDownDiv.addEventListener("animationstart", this.animationEventHandler, false);
    countDownDiv.addEventListener("animationend", this.animationEventHandler, false);
    countDownDiv.addEventListener("animationiteration", this.animationEventHandler, false);
  }

  animationEventHandler = (e) => {
    switch(e.type) {
      case "animationstart":
        // do nothing
        break;
      case "animationend":
        // do nothing
        break;
      case "animationiteration":
        if (this.state.countDownSeconds === this.props.goText) {
          this.props.onCountDownEnd && this.props.onCountDownEnd();
        } else {
          this.setState({
            countDownSeconds: (this.state.countDownSeconds <= 1 ? this.props.goText :
(this.state.countDownSeconds-1))
          }, () => {
            this.props.onCount && this.props.onCount(this.state.countDownSeconds);
          });
        }
        break;
      default:
        // do nothing
        break;
    }
  }

  beginCountDown = () => {
    this.setState({ animatedStyleClasses: "animated infinite zoomIn" });
  }

  render() {
    return (
      <div className="bounce-count-down-container" style={{display:(this.state.show?"flex":"none")}}>
        <div id="count-down-component" className={`count-down
${this.state.animatedStyleClasses}`}>{this.state.countDownSeconds}</div>
      </div>
    )
  }
}

let bounceCountDownContainerView = null;
BounceCountDown.show = (countDownSeconds, endCallback, counterCallback = null, goText = "GO") => {

```



```

    if (bounceCountDownContainerView) return;

    bounceCountDownContainerView = document.createElement('div');
    document.body.appendChild(bounceCountDownContainerView);

    const onCountDownEnd = () => {
      BounceCountDown.hide();
      endCallback();
    }

    ReactDOM.render((
      <BounceCountDown onCountDownEnd={onCountDownEnd} onCount={counterCallback} seconds={countDownSeconds}
      goText={goText} />
    ), bounceCountDownContainerView);
  };

  BounceCountDown.hide = () => {
    if (bounceCountDownContainerView) {
      ReactDOM.unmountComponentAtNode(bounceCountDownContainerView);
      document.body.removeChild(bounceCountDownContainerView);
      bounceCountDownContainerView = null;
    }
  }

  export default BounceCountDown;

import React from 'react';
import ReactDOM from 'react-dom';
import ActionButtonPanel from '../ActionButtonPanel';
import './GameEarningsPopup.scss';

class GameEarningsPopup extends React.Component {

  state = {
    show: false
  }

  componentDidMount() {
    setTimeout(() => this.present(), 0);
  }

  componentWillUnmount() {
    this.timer && clearTimeout(this.timer);
  }

  present = () => {
    this.setState({
      show: true
    }, () => {
      this.props.willPresent && this.props.willPresent();
      this.timer = setTimeout(() => {
        this.props.didPresent && this.props.didPresent();
      }, 300);
    });
  }

  dismiss = () => {
    this.setState({
      show: false
    }, () => {
      this.props.willDismiss && this.props.willDismiss();
      this.timer = setTimeout(() => {
        this.props.didDismiss && this.props.didDismiss();
      }, 300);
    });
  }
}

```

```

handleButtonClick = () => {
  if(this.props.content !==null){
    this.dismiss();
  }

  this.props.onButtonClick && this.props.onButtonClick();
}

render() {
  const showContent = this.props.content !==null;
  const win = showContent || this.props.resultInfo.earning > 0;
  return (
    <div className="game-earnings-popup">
      { /* background mask */ }
      <div className={`background-mask background-mask-fade-${this.state.show?"in":"out"}`} onClick={()
=> this.dismiss()} />
      { /* picker panel */ }
      <div className={`game-earnings-panel game-earnings-panel-${this.state.show?"present":"dismiss"}`}>
        <div className="float-panel">
          {
            win ? <img className="head-img-bingo" alt="" src={require("../images/game/bingo.png")} />
              : <img className="head-img-pity" alt="" src={require("../images/game/pity.png")} />
          }
          <div className="title">{ showContent ? this.props.resultInfo.title : (win ? "投注详情" : "很遗憾，本次投注没有中")}</div>
          {
            showContent ?
            <div className="betting-info">
              {this.props.content}
            </div>
            :
            <div className="betting-info">
              <div className="info-row">
                <div className="row-title">投注收益</div>
                <div className="row-value">{`¥${this.props.resultInfo.earning}`}</div>
              </div>
              <div className="info-row">
                <div className="row-title">投注期数</div>
                <div className="row-value">{this.props.resultInfo.bettingNumber}</div>
              </div>
              <div className="info-row">
                <div className="row-title">投注金额</div>
                <div className="row-value">{`¥${this.props.resultInfo.money.toFixed(0)}`}</div>
              </div>
              <div className="info-row">
                <div className="row-title">投注项目</div>
                <div className="row-value">{this.props.resultInfo.item}</div>
              </div>
              <div className="info-row">
                <div className="row-title">赔率</div>
                <div className="row-value">{`1赔${this.props.resultInfo.odds.toFixed(0)}`}</div>
              </div>
              <div className="info-row">
                <div className="row-title">标的股</div>
                <div className="row-
value">{`${this.props.resultInfo.stockInfo.name}(${this.props.resultInfo.stockInfo.date})`}</div>
              </div>
              <div className="info-row">
                <div className="row-title">涨跌幅</div>
                <div className="row-value">{`${this.props.resultInfo.opening.toFixed(2)}%`}</div>
              </div>
            </div>
          }
          <ActionButtonPanel style={{position: "absolute", backgroundColor: "transparent"}}>
            <ActionButtonPanel.Button onClick={this.handleButtonClick}>
              { showContent?this.props.resultInfo.buttonName: '查看详情' }
            </ActionButtonPanel.Button>
          </ActionButtonPanel>
        </div>
      </div>
    </div>
  );
}

```

```

        </ActionButtonPanel.Button>
      </ActionButtonPanel>
    </div>
  </div>
</div>
)
}
}

let gameEarningsPopupContainerView = null;
GameEarningsPopup.show = (resultInfo, onButtonClick) => {

  if (gameEarningsPopupContainerView) return;

  gameEarningsPopupContainerView = document.createElement('div');
  document.body.appendChild(gameEarningsPopupContainerView);

  const onDismiss = () => {
    ReactDOM.unmountComponentAtNode(gameEarningsPopupContainerView);
    document.body.removeChild(gameEarningsPopupContainerView);
    gameEarningsPopupContainerView = null;
  }

  const onButtonClickCallback = (matchItem) => {
    ReactDOM.unmountComponentAtNode(gameEarningsPopupContainerView);
    document.body.removeChild(gameEarningsPopupContainerView);
    gameEarningsPopupContainerView = null;
    onButtonClick && onButtonClick(matchItem);
  }

  ReactDOM.render((
    <GameEarningsPopup content={resultInfo.content} resultInfo={resultInfo} didDismiss={onDismiss}
    onButtonClick={onButtonClickCallback} />
  ), gameEarningsPopupContainerView);
};

export default GameEarningsPopup;

import React from 'react';
import CloseIcon from 'react-icons/lib/ti/delete';
import KlineChart from '../charts/KlineChart';
import './KlineChartModalView.scss';

class KlineChartModalView extends React.Component {

  resizeChart = () => {
    this.klineChart && this.klineChart.resize();
  }

  render() {
    return (
      <div className={`kline-chart-modal-view ${this.props.show?"":"kline-chart-modal-view-fade-out"}`>
        <div className={this.props.onClose} style={this.props.style} >
          <div className={`kline-chart-wrapper ${this.props.show?"slide-in":"slide-out"}`>
            <div className="close-button" onClick={this.props.onClose}>
              <CloseIcon />
            </div>
            <div className="header">
              系统从真实历史行情中选取某一股票的一个交易日进行竞赛，以下是此股票当日开盘前的日K线图。
            </div>
            <div className="chart">
              { this.props.historyData && <KlineChart historyData={this.props.historyData} ref={node =>
                (this.klineChart = node)} /> }
            </div>
          </div>
        </div>
      </div>
    )
  }
}

```

```

}

export default KlineChartModalView;

import history from './history';
import CommonUtil from '../utils/CommonUtil';

export default {
  navigateToLoginView: (refer) => {
    history.push("/login?refer=" + refer);
  },
  navigateToHomeView: () => {
    history.push(`/home`);
  },
  navigateToProfileView: (state) => {
    history.push(`/profile`, state);
  },
  navigateToContestHomeView: (replace = false, showHUB = false) => {
    if (replace) {
      history.replace(`/contest${showHUB?"?share=1":""}`);
    } else {
      history.push(`/contest${showHUB?"?share=1":""}`);
    }
  },
  navigateToPrivateContestRoomView: (state, replace = false) => {
    history.push(`/contest/private/${state.contestId}`, state);
  },
  navigateToContestView: (state, replace = false, watch = false) => {
    const url = `/match/${state.contestId}${watch?"?watchOnly=1":""}`;
    if (replace) {
      history.replace(url, state);
    } else {
      history.push(url, state);
    }
  },
  navigateToContestResultView: (state, replace = false, watch = false, share = false) => {
    let url = `/contest/result/${state.contestId}`;
    if (watch) {
      url = CommonUtil.appendParamToUrl(url, "watch", 1);
    }
    if (share) {
      url = CommonUtil.appendParamToUrl(url, "share", 1);
    }
    if (replace) {
      history.replace(url, state);
    } else {
      history.push(url, state);
    }
  },
  navigateToContestStatProfileView: (userId, state, replace = false) => {
    let url = `/contest/stat/${userId}`;
    if (replace) {
      history.replace(url, state);
    } else {
      history.push(url, state);
    }
  },
  navigateToContestReplayView: (contestId) => {
    const url = `/contest/replay/${contestId}`;
    history.push(url);
  },
  navigateToContestHistoryView: () => {
    const url = `/contest/history`;
    history.push(url);
  },
  navigateToKlinePracticeView: (replace = false) => {
    const url = `/practice/kline`;
  }
}

```

```

    if (replace) {
      history.replace(url);
    } else {
      history.push(url);
    }
  },
  navigateToKlineCategoryPracticeView: (id) => {
    history.push(`/training/kpattern?id=${id}`);
  },
  navigateToTrainingView: (replace = false, category = null) => {
    const url = category ? `/training/technical/${category}` : `/training`;
    if (replace) {
      history.replace(url);
    } else {
      history.push(url);
    }
  },
  navigateToTechTrainingPracticeView: (category, id, replace = false) => {
    const url = `/training/${category}/${id}/practice`;
    if (replace) {
      history.replace(url);
    } else {
      history.push(url);
    }
  },
  navigateToTechTrainingItemPresentationView: (category, id, replace = false, queryStr = "") => {
    const url = `/training/${category}/${id}${queryStr}`;
    if (replace) {
      history.replace(url);
    } else {
      history.push(url);
    }
  },
  navigateToPracticeView: (id = '', replace = false) => {
    if (id !== '') {
      if (replace) {
        history.replace(`/practice?id=${id}`);
      } else {
        history.push(`/practice?id=${id}`);
      }
    } else {
      if (replace) {
        history.replace(`/practice`);
      } else {
        history.push(`/practice`);
      }
    }
  },
  navigateToPracticeResultView: (id = '', trainId = '', replace = false) => {
    if (replace) {
      history.replace(`/training/result/${trainId}?id=${id}`);
    } else {
      history.push(`/training/result/${trainId}?id=${id}`);
    }
  },
  navigateToRankingView: () => {
    history.push(`/ranking`);
  },
  navigateToBetGameView: () => {
    history.push(`/game/bet`);
  },
  navigateToBetGameHistoryView: () => {
    history.push(`/game/bet/history`);
  },
  navigateToGameResultView: (id = "") => {
    history.push(`/game/bet/result/${id}`);
  },
  navigateToFavoriteStocksView: () => {
    history.push(`/similar-kline/favorites`);
  },
  navigateToStockMatchingView: (favoriteStocks) => {

```

```

    history.push(`/similar-kline/stock-matching`, {favoriteStocks});
  },
  navigateToSimilarKlineView: (stockCode = '', translate = true, state) => {
    if (translate) {
      stockCode = stockCode.toUpperCase();
      if (stockCode.indexOf("SZ") >= 0) {
        stockCode = stockCode.replace(new RegExp("SZ"), "");
        stockCode = stockCode + ".SZ";
      } else if (stockCode.indexOf("SH") >= 0) {
        stockCode = stockCode.replace(new RegExp("SH"), "");
        stockCode = stockCode + ".SH";
      }
    }
    history.push(`/similar-kline/${stockCode}`, state);
  },
  navigateToSimilarKlinePracticeView: (stockCode = '', similarStockCode = '', similarKlineData = {},
  sourceKlineData = {}, sourceStockName = '') => {
    history.push(`/similar-kline/practice/${stockCode}/${similarStockCode}`, { similarKlineData,
    sourceKlineData, sourceStockName });
  },
  navigateToSimilarKlinePracticeResultView: (stockCode = '', similarStockCode = '', similarKlineData = {},
  sourceKlineData = {}, sourceStockName = '', yieldRate = 0.0, actions = []) => {
    history.replace(`/similar-kline/result/${stockCode}/${similarStockCode}`, { similarKlineData,
    sourceKlineData, yieldRate, actions, sourceStockName });
  },
  navigateToStockGameHomeView: () => {
    history.push(`/stock-game`, { });
  },
  navigateToStockGameView: ({gameId}) => {
    history.push(`/stock-game/${gameId}`, { });
  },
  push: (path) => {
    history.push(path);
  },
  replace: (path) => {
    history.replace(path);
  },
  back: () => {
    history.goBack();
  }
}

```

```

import random from 'lodash/random';
import shuffle from 'lodash/shuffle';
import IndicatorPracticeApi from '../api/IndicatorPracticeApi';
import Logger from '../utils/Logger';

```

```

const SAMPLE_SIZE = 40;
const NUM_BARS_TRADE = 10;
const NUM_BARS_DIFF = 20;

```

```
/**
```

```
 * 获取指标信号区分练习
```

```
 * @param {object} data
```

```
 * @param {string} indicatorId
```

```
 */
```

```
function getDiffPractice(data, indicatorId) {
```

```
  const diffPractice = {};
```

```
  let numBarsForOption = NUM_BARS_DIFF;
```

```
  const keys = [];
```

```
  const lastIndex = data.puzzle.bars.timestamp.length-1;
```

```
  const signals = data.puzzle.indicators[indicatorId];
```

```
  for(let k in signals) {
```

```
    keys.push(k);
```

```
  }
```

```

keys.sort((a,b)=>a-b);

const ranges = [];
//answer,所有选项长度固定为: numBarsForOption
const i1 = random(keys.length-1);
const signalIndex = window.parseInt(keys[i1]);
const r1 = {
  isAnswer:true,
  signalIndex:signalIndex,
  start_index: Math.max(signalIndex-numBarsForOption/2,0),
  end_index: Math.min(signalIndex+numBarsForOption/2,lastIndex)
};
ranges.push(r1);

//wrong options, 不可以出现信号
//在两个信号范围内查找
for(let i=0; i<keys.length-1; i++) {
  const signalIndex2 = window.parseInt(keys[i]);
  const _signalIndex2 = window.parseInt(keys[i+1]);

  if(_signalIndex2 - signalIndex2 <= numBarsForOption) {
    continue;
  }

  const r2 = {
    start_index: signalIndex2 + 1,
    end_index: Math.min(signalIndex2+numBarsForOption+1, _signalIndex2-1)
  };
  ranges.push(r2);

  if(ranges.length>=3) {
    break;
  }
}

//在最大最小两个信号两端范围外查找
if(ranges.length<3) {
  const indicatorMinIndex = window.parseInt(keys[0]);
  const indicatorMaxIndex = window.parseInt(keys[keys.length-1]);
  const len = 3-ranges.length;
  for(let i=0; i<len; i++) {
    let start_index = 0;
    if(indicatorMinIndex<numBarsForOption*2) {
      start_index = random(indicatorMaxIndex, lastIndex-numBarsForOption);
    } else {
      start_index = random(0, indicatorMinIndex-numBarsForOption);
    }
    ranges.push({start_index, end_index: start_index+numBarsForOption});
  }
}

const options = shuffle(ranges).map((range, index) => {
  if (range.isAnswer) {
    diffPractice.answer = index;
  }

  const option = {};
  for(let key in data.puzzle.bars) {
    option[key] = data.puzzle.bars[key].slice(range.start_index, range.end_index);
  }
  return option;
});
diffPractice.options = options;

return diffPractice;
}

/**

```

```

* 获取指标信号识别练习
* @param {object} data
* @param {string} indicatorId
*/
function getRecogPractice(data, indicatorId) {
  const recogPractice = {};
  const signals = data.puzzle.indicators[indicatorId];

  const keys = [];
  for(let k in signals) {
    keys.push(k);
  }
  keys.sort((a,b)=>a-b);

  const index = random(0,keys.length-1);
  const key = keys[index];
  const signalIndex = window.parseInt(key);

  let offset=0,from=0,to=0;
  offset = random(Math.min(SAMPLE_SIZE - 1, signalIndex));
  from = signalIndex - offset;
  to = from + SAMPLE_SIZE;

  recogPractice.orginSignalIndex = signalIndex;
  recogPractice.signalIndex = signalIndex - from;
  recogPractice.answers = [];
  for(let i=0; i<keys.length; i++){
    const sIndex = window.parseInt(keys[i]);
    if(sIndex>=from && sIndex <= to) {
      const answerSignalIndex = sIndex - from;
      recogPractice.answers.push(answerSignalIndex);
    }
  }

  recogPractice.from = from;
  recogPractice.to = to;

  recogPractice.bars={};
  for(let key in data.puzzle.bars) {
    recogPractice.bars[key] = data.puzzle.bars[key].slice(from, to);
  }

  return recogPractice;
}

/**
* 获取指标信号交易练习
* @param {object} data
* @param {string} indicatorId
*/
function getTradingPractice(data, indicatorId) {
  const practice = {bars:{}};
  const signals = data.puzzle.indicators[indicatorId];
  const keys = [];
  for(let k in signals) {
    keys.push(k);
  }
  keys.sort((a,b)=>a-b);

  const index = random(0,keys.length-1);
  const key = keys[index];
  const signalIndex = window.parseInt(key);
  practice.orginSignalIndex = signalIndex;

  const to = signalIndex + NUM_BARS_TRADE + 1;
  let from = to - SAMPLE_SIZE;
  if(from < 0 ) {
    from = signalIndex;
  }

```



```

    }

    for(let key in data.puzzle.bars) {
        practice.bars[key] = data.puzzle.bars[key].slice(from, to);
    }

    practice.indicatorId = indicatorId;
    practice.signalIndex = to - from - NUM_BARS_TRADE - 1;

    practice.from = from;
    practice.to = to;

    return practice;
}

const IndicatorPracticeService = {
    getNewIndicatorPractice: ({
        indicatorId
    }) => {
        return IndicatorPracticeApi.getNewIndicatorPractice({
            indicatorId
        }).then((res) => {
            let data = res;

            const diffPractice = getDiffPractice(data, indicatorId);
            const recogPractice = getRecogPractice(data, indicatorId);
            const tradingPractice = getTradingPractice(data, indicatorId);

            const indicatorPractice = {
                practiceId: data.practiceId,
                indicator: data.indicator,
                diffPractice,
                recogPractice,
                tradingPractice,
            };
            return indicatorPractice;
        }).catch((res) => {
            Logger.error("[IndicatorPracticeService] get new IndicatorPractice failed: " + JSON.stringify(res));
            throw res;
        });
    },

    saveIndicatorPractice: (params) => {
        return IndicatorPracticeApi.saveIndicatorPractice(params).then((res) => {
            return res.data;
        }).catch((res) => {
            Logger.error("[IndicatorPracticeService] saveIndicatorPractice failed: " + JSON.stringify(res));
        });
    },
};

export default IndicatorPracticeService;

import TrainApi from '../api/TrainApi';
import Logger from '../utils/Logger';

// TODO: 暂用leancloud存储练习统计数据, 后面可能要改成我们自己的后台显示
// import LeanCloudStorage from '../utils/LeanCloudStorage';

const TrainingService = {
    getNewTrain: (id) => {
        return TrainApi.getNewTrain(id).then((res) => {
            let trainData = res;
            return trainData;
        }).catch((res) => {
            Logger.error("[TrainingService] get new train failed: " + JSON.stringify(res));
        });
    };
};

```

```

},

getTrainingCount: (params) => {
  return TrainApi.getTrainingCount(params).then((res) => {
    let trainData = res;
    return trainData;
  }).catch((res) => {
    Logger.error("[TrainingService] get training count failed: " + JSON.stringify(res));
  });
},

getTrainingResult: (trainingId) => {
  return TrainApi.getTrainingResult(trainingId).then((res) => {
    let trainingResult = res;
    return trainingResult;
  }).catch((res) => {
    Logger.error("[TrainingService] get training result failed: " + JSON.stringify(res));
  });
},

getKlineCategoriesData: () => {
  return new Promise((resolve, reject) => {
    const categoriesData = require('../mock/klineCategoriesData.json');
    resolve(categoriesData);
    // MODIFIED: 2017/12/23, 全部解锁
    // TrainingService.getTrainingStats().then((stats) => {
    //   let preWinRate = 0;
    //   let preFinishedCount = 0;
    //   categoriesData.forEach((item, index) => {
    //     const itemStat = stats[item.patternId] || null;
    //     if (itemStat) {
    //       item.finishedCount = itemStat.finishedCount;
    //       item.winCount = itemStat.winCount;
    //       item.winRate = ((item.winCount / item.finishedCount) * 100).toFixed(0);
    //     } else {
    //       item.finishedCount = 0;
    //       item.winCount = 0;
    //       item.winRate = 0;
    //     }
    //     item.locked = (index > 4 && (preWinRate < 60 || preFinishedCount < 20));
    //     preWinRate = item.winRate;
    //     preFinishedCount = item.finishedCount;
    //   });
    //   resolve(categoriesData);
    // }).catch((error) => {
    //   reject(error);
    // });
  })
},

// getTrainingStats: () => {
//   return new Promise((resolve, reject) => {
//     LeanCloudStorage.getPracticeStats().then((practiceStats) => {
//       resolve((practiceStats.get('stats') || {}));
//     }).catch((error) => {
//       reject(error);
//     });
//   });
// },

addTrainingStat: (id, win) => {
  return new Promise((resolve, reject) => {

    TrainingService.getTrainingStats().then((stats) => {

      let itemStat = stats[id] || null;

      if (itemStat) {
        itemStat.finishedCount = itemStat.finishedCount + 1;

```

```

        itemStat.winCount = itemStat.winCount + (win?1:0);
    } else {
        itemStat = {
            finishedCount: 1,
            winCount: (win?1:0)
        };
    }

    // LeanCloudStorage.updateStat(id, itemStat).then(() => {
    //     resolve();
    // }).catch((error) => {
    //     reject(error);
    // });
    }).catch((error) => {
        reject(error);
    });
    });
},

// ===== 获取练习统计数据 =====

getPracticeStats: () => {
    return TrainApi.getPracticeStats().then((stats) => {
        return stats;
    }).catch((res) => {
        Logger.error("[TrainingService] get practice stat failed: " + JSON.stringify(res));
    });
},

// ===== 新形态训练 =====

getPatternsMap: () => {
    return TrainApi.getPatternsMap().then((patternsMap) => {
        return patternsMap;
    }).catch((res) => {
        Logger.error("[TrainingService] get patterns map failed: " + JSON.stringify(res));
    });
},

getPatternDetails: (patternId) => {
    return TrainApi.getPatternDetails(patternId).then((details) => {
        return details;
    }).catch((res) => {
        Logger.error("[TrainingService] get pattern details failed: " + JSON.stringify(res));
    });
},

// ===== 新形态训练 =====

getIndicatorsMap: () => {
    return TrainApi.getIndicatorsMap().then((indicatorsMap) => {
        return indicatorsMap;
    }).catch((res) => {
        Logger.error("[TrainingService] get indicators map failed: " + JSON.stringify(res));
    });
},

getIndicatorDetails: (indicatorId) => {
    return TrainApi.getIndicatorDetails(indicatorId).then((details) => {
        return details;
    }).catch((res) => {
        Logger.error("[TrainingService] get indicator details failed: " + JSON.stringify(res));
    });
}
}

export default TrainingService;

```

```

const useDefaultHeadImgIfNull = (headImgUrl) => {
  return headImgUrl ? headImgUrl :
  "https://www.timeshighereducation.com/sites/default/files/byline_photos/default-avatar.png";
}

const appendParamToUrl = (url, key, value) => {
  if (key && value) {
    return `${url}${url.indexOf('?') >= 0 ? '&':'?'}${key}=${value}`;
  }
  return url;
};

const getCurrentLinkWithoutQueryString = () => {
  return window.location.href.replace(window.location.search, '');
}

const getDPR = () => {
  let isMobile = false;
  if ((/android/i).test(navigator.userAgent) || (/iphone|ipad/i).test(navigator.userAgent)) {
    const meta = document.querySelector('meta[name="viewport"]');
    const content = meta.content;
    if (!(/initial-scale=1,/g.test(content))) {
      isMobile = true;
    }
  }
  return (isMobile ? (window.devicePixelRatio || 1) : 1);
}

const queryString = require('query-string');
const parseParams = (string = '') => {
  return queryString.parse(string);
}

const toDecimal2 = (x) => {
  var f = parseFloat(x);
  if (isNaN(f)) {
    return false;
  }
  f = Math.round(x*100)/100;
  var s = f.toString();
  var rs = s.indexOf('.');
  if (rs < 0) {
    rs = s.length;
    s += '.';
  }
  while (s.length <= rs + 2) {
    s += '0';
  }
  return s;
}

const isOpenedFromShare = (location) => {
  if (typeof location == 'string') {
    return location && location.indexOf('share=1') >= 0;
  }

  if (location.pathname === '/home' || location === '/') {
    return false;
  }
  return location && location.search && location.search.indexOf('share=1') >= 0;
}

Date.prototype.format = function(format) {
  let date = {
    "M+": this.getMonth() + 1,
    "d+": this.getDate(),
  }

```

```

    "h+": this.getHours(),
    "m+": this.getMinutes(),
    "s+": this.getSeconds(),
    "q+": Math.floor((this.getMonth() + 3) / 3),
    "S+": this.getMilliseconds()
  };
  if (/ (y+)/i.test(format)) {
    format = format.replace(RegExp.$1, (this.getFullYear() + '').substr(4 - RegExp.$1.length));
  }
  for (let k in date) {
    if (new RegExp("(" + k + ")").test(format)) {
      format = format.replace(RegExp.$1, RegExp.$1.length === 1
        ? date[k] : ("00" + date[k]).substr(("" + date[k]).length));
    }
  }
  return format;
}
let formatterDate = new Date();
const getFormatDateString = (timestamp, format = 'yyyy-MM-dd') => {
  formatterDate.setTime(timestamp);
  return formatterDate.format(format);
}

const getScreenPosition = (object) => {
  let position = {};
  position.x = object.offsetLeft;
  position.y = object.offsetTop;

  while (object.offsetParent) {
    position.x = position.x + object.offsetParent.offsetLeft;
    position.y = position.y + object.offsetParent.offsetTop;

    if (object === document.getElementsByTagName("body")[0]) {
      break;
    }
    else {
      object = object.offsetParent;
    }
  }
  return position;
}

export default {
  useDefaultHeadImgIfNull,
  appendParamToUrl,
  getCurrentLinkWithoutQueryString,
  getDPR,
  parseParams,
  toDecimal2,
  isOpenedFromShare,
  getFormatDateString,
  getScreenPosition
};

import AV from 'leancloud-storage';
import SessionService from '../service/SessionService';

const LEANCLOUD_APP_ID = 'msdkg8vEjSJQ8uwCNT1hYmuV-gzGzoHsz';
const LEANCLOUD_APP_KEY = 'Qs3jKECqlzym6Sh7CisMXDNF';

let _practiceStats = null;

const init = () => {
  AV.init({
    appId: LEANCLOUD_APP_ID,
    appKey: LEANCLOUD_APP_KEY
  });
};

```

```

};

const defaultStats = (userId) => {
  let PracticeStats = AV.Object.extend('PracticeStats');
  let newStats = new PracticeStats();
  newStats.set('userId', userId);
  newStats.set('stats', {});
  return newStats;
};

const getPracticeStats = () => {
  return new Promise((resolve, reject) => {

    if (_practiceStats) {
      resolve(_practiceStats);
      return;
    }

    if (!(SessionService.isLoggedIn())) {
      reject(new Error(1000, "needs login"));
      return;
    }

    let query = new AV.Query('PracticeStats');
    let userId = SessionService.getLoggedInUserInfo().userId;
    query.equalTo('userId', userId);
    query.find().then((results) => {
      if (results.length) {
        _practiceStats = results[0];
        resolve(_practiceStats);
      } else {
        _practiceStats = defaultStats(userId);
        resolve(_practiceStats);
      }
    }, function (error) {
      console.error(error);
      reject(error);
    });

  });
};

const updateStat = (categoryId, statInfo = {}) => {
  return new Promise((resolve, reject) => {
    getPracticeStats().then((practiceStats) => {
      let stats = practiceStats.get('stats') || {};
      stats[categoryId] = statInfo;
      practiceStats.set('stats', stats);
      practiceStats.save().then((stat) => {
        resolve();
      }, function (error) {
        console.error(error);
        reject(error);
      });
    });
  });
};

export default {
  init,
  getPracticeStats,
  updateStat
};

const Cookies = require('js-cookie');

const cookieDAO = {

```

```

    getUsername: () => {
      return Cookies.get('username') || '';
    }
  }

  const isLocalStorageAvailable = () => {
    return typeof localStorage !== 'undefined' && localStorage !== null;
  }

  const localStorageDAO = {
    KEYS: {
      RECENT_STOCK_SEARCHES: "RECENT_STOCK_SEARCHES",
    },
    retrieveObject: (key) => {
      if (key && isLocalStorageAvailable()) {
        var value = localStorage.getItem(key);
        if (!value) {
          return null;
        }
        else {
          return JSON.parse(value);
        }
      }
      return null;
    },
    storeObject: (key, object) => {
      if (isLocalStorageAvailable() && key && object) {
        try {
          let value = JSON.stringify(object);
          localStorage.setItem(key, value);
          return true;
        } catch(e) {
          console.error("error when storing value to local storage: " + JSON.stringify(e));
        }
      }
      return false;
    },
    removeValue: (key) => {
      if (isLocalStorageAvailable() && key) {
        try {
          localStorage.removeItem(key);
          return true;
        } catch(e) {
          console.error("error when removing value from local storage: " + JSON.stringify(e));
        }
      }
      return false;
    }
  }

  export default {
    cookieDAO,
    localStorageDAO
  };

import React, { Component } from 'react';
import SessionService from '../service/SessionService';
import ViewContainer from '../skeleton/ViewContainer';
import UIDisplayingHelper from '../component/helper/UIDisplayingHelper';
import navigator from '../routes/navigator';
import ProfileHeader from './ProfileHeader';
import Tiles from './Tiles';
import './HomeView.scss';

class HomeView extends Component {

  state = {

```

```

    username: '',
    nickname: '',
    headImgUrl: '',
    balance: 0,
    score: 0,
  }

  componentDidMount() {
    this.showNavBar(false);
    this.setTitle("小Q教练");
    this.setState({
      username: SessionService.getLoggedInUsername(),
      nickname: SessionService.getLoggedInNickname(),
      headImgUrl: SessionService.getLoggedInUserHeadImgUrl(),
      balance: SessionService.getLoggedInUserMoney(),
      score: SessionService.getLoggedInUserScore()
    });
  }

  // TODO: remove simulate loading, just for demo purpose at staging phase
  simulateLoadingThen = (action) => {
    UIDisplayingHelper.showLoading();
    setTimeout(() => {
      UIDisplayingHelper.hideLoading();
      action && action();
    }, 300);
  }

  handleContestTileClick = () => {
    this.simulateLoadingThen(() => {
      navigator.navigateToContestHomeView();
    });
  }

  handlePracticeTileClick = () => {
    this.simulateLoadingThen(() => {
      navigator.navigateToTrainingView();
    });
  }

  handleSimilarKlineTileClick = () => {
    this.simulateLoadingThen(() => {
      navigator.navigateToFavoriteStocksView();
    });
  }

  handleGameTileClick = () => {
    this.simulateLoadingThen(() => {
      navigator.navigateToBetGameView();
    });
  }

  handleNewGameClick = () => {
    this.simulateLoadingThen(() => {
      navigator.navigateToStockGameHomeView();
    });
  }

  handleProfileHeaderClick = () => {
    this.simulateLoadingThen(() => {
      navigator.navigateToProfileView(SessionService.getLoggedInUserInfo());
    });
  }

  unopened = () => {
    UIDisplayingHelper.showToast('该功能暂未开放')
  }

  render() {

```



```

const isTrial = SessionService.getLoggedInUsername().indexOf('TR_')===0;
//const isTrial = false;

/* global __VERSION__:false */
const version = __VERSION__;
return (
  <ViewContainer style={{display: 'flex', flexFlow: 'column'}}>
    <div className="home-view home-page">
      <ProfileHeader headImgUrl={this.state.headImgUrl}
        nickname={this.state.nickname}
        balance={this.state.balance}
        score={this.state.score}
        onClick={this.handleProfileHeaderClick} />
      <Tiles>
        <Tiles.Item icon={require('../../../../images/home/contest_icon.png')}
          onClick={this.handleContestTileClick}>
          >交易竞赛</Tiles.Item>
        <Tiles.Item className={isTrial?'unopened':''}
          icon={require('../../../../images/home/training_icon.png')}
          onClick={isTrial?this.unopened:this.handlePracticeTileClick}>
          >交易训练</Tiles.Item>
        <Tiles.Item className={isTrial?'unopened':''}
          icon={require('../../../../images/home/similar_kline_icon.png')}
          onClick={isTrial?this.unopened:this.handleSimilarKlineTileClick}>
          >相似K线</Tiles.Item>
        <Tiles.Item className={isTrial?'unopened':''}
          icon={require('../../../../images/home/game_icon.png')}
          onClick={isTrial?this.unopened:this.handleGameTileClick}>
          >涨跌猜猜乐</Tiles.Item>

        {/* !isTrial && <div className="new-game"
          onClick={isTrial?this.unopened:this.handleNewGameClick}>新游戏 (beta) </div> */}
      </Tiles>

      <div className="xmas-snow" style={{left: '50%', animationDelay: '0.5s'}}/>
      <div className="xmas-snow" style={{left: '20%', animationDelay: '3s'}}/>
      <div className="xmas-snow" style={{left: '30%', animationDelay: '6s'}}/>
      <div className="xmas-snow" style={{left: '80%', animationDelay: '8s'}}/>

      <div className="t-version" >{version}</div>
    </div>
  </ViewContainer>
);
}
}

export default HomeView;

import React, { Component } from 'react';
import PropTypes from 'prop-types';
import QRCodeIcon from 'react-icons/lib/fa/qrcode';

class KlineChartPanel extends Component {
  render() {
    return (
      <header className="home-view-header">
        {/*<div className="qr-code">*/}
        <QRCodeIcon className="qr-code" onClick={this.showShareLayer} />
        {/*</div>*/}
        <div className="info-wrapper animated swing left" onClick={() => {this.props.onClick &&
this.props.onClick()}}>
          <span className="title">我的财富</span>
          <span className="value">¥{this.props.balance}</span>
        </div>
        <div className="profile-info-wrapper" onClick={() => {this.props.onClick &&
this.props.onClick()}}>

```

```

        <span className="avatar">
          <img alt="" src={this.props.headImgUrl} />
          <span className="name">{this.props.nickname}</span>
        </span>
      </div>
      <div className="info-wrapper animated swing right" onClick={() => {this.props.onClick &&
this.props.onClick()}}>
        <span className="title">Score</span>
        <span className="value">{this.props.score.toFixed(2)}</span>
      </div>
    </header>
  );
}
}

KlineChartPanel.propTypes = {
  balance: PropTypes.number,
  headImgUrl: PropTypes.string,
  nickname: PropTypes.string,
  score: PropTypes.number
}

export default KlineChartPanel;

import React, { Component } from 'react';
import PropTypes from 'prop-types';

class TileItem extends Component {
  render() {
    return (
      <div className={`card ${this.props.className}`} onClick={this.props.onClick}>
        <img className="icon" alt="" src={this.props.icon}/>
        <span className="title">{this.props.children}</span>
      </div>
    );
  }
}

TileItem.propTypes = {
  onClick: PropTypes.func,
  icon: PropTypes.string
}

class Tiles extends Component {
  render() {
    return (
      <div className="home-view-cards-wrapper">
        {this.props.children}
      </div>
    );
  }
}

Tiles.Item = TileItem;

export default Tiles;

import React, { Component } from 'react';
import { Prompt } from 'react-router-dom';
import ChartBar from 'react-icons/lib/ti/chart-bar';
import ChartLine from 'react-icons/lib/ti/chart-line';
import StompClient from '../../utils/StompClient';
import TrainingService from '../../service/TrainingService';
import CommonUtil from '../../utils/CommonUtil';
import Contants from '../../common/Contants';

```

```

import navigator from '.././././routes/navigator';
import UIDisplayingHelper from '.././././component/helper/UIDisplayingHelper';
import ViewContainer from '.././././skeleton/ViewContainer';
import ActionButtonPanel from '.././././component/ActionButtonPanel';
import TimingChart from '.././././charts/TimingChart/index';
import BounceCountDown from '.././././component/BounceCountDown/index';
import KlineChartModalView from '.././././component/KlineChartModalView/index';
import TradingHintPopup from '.././././component/TradingHintPopup';
import calYieldRate from '.././././charts/TimingChart/calYieldRate';
import HeaderBanner from '.././././component/ContestView/HeaderBanner/index';
import BottomBanner from '.././././component/ContestView/BottomBanner/index';
import './PracticeView.scss';

class PracticeView extends Component {

  state = {
    trainingInfo: null, // 练习信息 (trainingId, createTime, interval, history etc) , 后台返回
    playing: false, // 交易进行中
    curIndex: -1, // 当前Index
    curBarItem: {}, // 分时图当前Index的数据 (当需要获取当前价格或macd等数据时有用)
    yieldRate: 0, // 当前收益率
    curTradeAction: Contants.TRADE_ACTIONS.CLOSE, // 当前交易动作, 初始平仓 (空仓), 点买即在买入状态, 点卖即在
    // 卖出状态 (影响收益计算和交易按钮可用状态)
    tradingActions: [], // 交易动作序列 (计算收益和买卖标注点/标注区域用)
    countDownSeconds: 10, // 倒计时时间, 后台返回, 给默认初始值10秒
    currentVolumes: 0, // 当前时间总成交量
    currentHighestPrice: 0, // 当前时间最高价
    currentLowestPrice: 0, // 当前时间最低价
    showKlineModalView: true, // 控制K线面板显示
    displayNoneKlineModalView: false,
    timingChartStyle: 'timing'
  }

  // 取消订阅回调数组, 界面销毁时调用
  unsubscribes = []

  // View life cycle: Mount
  componentDidMount() {
    this.showNavBar(false);
    this.getNewTrainingPuzzleThenBegin();
  }

  // View life cycle: Unmount
  componentWillUnmount() {
    // 取消消息订阅
    this.unsubscribes && this.unsubscribes.forEach((unsub) => {
      unsub.unsubscribe();
    });
    this.unsubscribes = null;
    // 定时器清除
    this.tickTimer && clearTimeout(this.tickTimer);
    this.silentCountDown && clearInterval(this.silentCountDown);
    this.silentCountDown = null;
    BounceCountDown.hide();
  }

  // 获取url传参
  getQueryParam = (paramKey) => {
    let queryParams = CommonUtil.parseParams(this.props.location.search);
    return queryParams[paramKey] || '';
  }

  // 获取练习题数据, 然后开始
  getNewTrainingPuzzleThenBegin = () => {
    const patternId = this.getQueryParam('id');

```

```

    this.setTitle(`形态练习`);

    const newTraining = TrainingService.getNewTrain;
    const param = patternId;

    UIDisplayingHelper.showLoading();
    newTraining.call(this, param).then(({ trainId }) => {
        UIDisplayingHelper.hideLoading();
        this.subscribeTrainingMessages(trainId);
    }).catch(() => {
        UIDisplayingHelper.hideLoading();
        UIDisplayingHelper.showToast("获取练习题目时发生错误(0)");
    });
}

// 监听练习数据回调消息 // TODO: 目前没有对监听消息未回调, 或者错误, 或链接断开的异常情况做处理, 务必记得加上
subscribeTrainingMessages = (trainId) => {

    // 练习题基本信息&历史K线数据
    UIDisplayingHelper.showLoading("接收交易数据中");
    StompClient.subscribe(`/user/queue/train/${trainId}`,
        (subs) => {
            // 记住取消订阅函数, 界面销毁时取订
            this.unsubscribes.push(subs);
        },
        (data) => {
            UIDisplayingHelper.hideLoading();

            // 练习信息 trainingId, createTime, history etc
            const trainingInfo = data;
            this.setState({
                trainingInfo: trainingInfo,
            }, () => {
                setTimeout(() => {
                    this.countDownThenBegin();
                }, 0);
            });

            // 初始化分时数据序列, 后面timetrend订阅收到的数据将存入其中。不放在state里, 是因为需要频繁塞入数据变更。
            this.timingData = {
                preClosePrice: trainingInfo.puzzle.oneDay.preClose,
                barTotal: trainingInfo.puzzle.oneDay.barTotal,
                barList: [], // 后台返回的原始分序列数据
                timestamps: Array(trainingInfo.puzzle.oneDay.barTotal).fill(0), // 分时图时间序列
                prices: [], // 分时图价格序列
                averagePrices: [], // 分时图均价序列
                volumes: [], // 分时图成交量序列
                volumesWithFlag: [], // 分时图成交量序列, 包含涨跌标记
                macd: [], // 分时图macd序列
                dea: [], // 分时图dea序列
                diff: [], // 分时图diff序列
                klineData: []
            };
        }
    );

    // 练习题分时走势数据
    StompClient.subscribe(`/user/queue/train/${trainId}/timetrend`,
        (subs) => {
            // 记住取消订阅函数, 界面销毁时取订
            this.unsubscribes.push(subs);
        },
        (data) => {
            // 异常处理
            if (!(this.timingData)) {

```

```

    return;
  }

  const barItem = data;
  // 保留一份后台返回的原始队列数据
  this.timingData.barList[barItem.barIndex] = barItem;
  // 时间序列
  this.timingData.timestamps[barItem.barIndex] = barItem.timestamp;
  // 价格序列
  this.timingData.prices[barItem.barIndex] = barItem.close;
  // 均价序列
  this.timingData.averagePrices[barItem.barIndex] = barItem.ma;
  // 成交量序列
  this.timingData.volumes[barItem.barIndex] = barItem.volume;
  // 成交量序列, 包含涨跌标记 [index, volume, flag(1/-1)]
  const upColor = barItem.barIndex > 0 && barItem.close > this.timingData.prices[barItem.barIndex-1];
  this.timingData.volumesWithFlag[barItem.barIndex] = [barItem.barIndex, barItem.volume, upColor?1:-1];

  // macd序列
  this.timingData.macd[barItem.barIndex] = barItem.macd;
  // dea序列
  this.timingData.dea[barItem.barIndex] = barItem.dea;
  // diff序列
  this.timingData.diff[barItem.barIndex] = barItem.diff;
  // K线序列
  const open = barItem.barIndex > 0 ? this.timingData.barList[barItem.barIndex-1].close : barItem.open;
  this.timingData.klineData.push([
    open,
    barItem.close,
    Math.min(open, barItem.close)*((Math.random()*3-3)*(0.001)+1.0),
    Math.max(open, barItem.close)*((Math.random()*3)*(0.001)+1.0)
  ]);
}
);

StompClient.subscribe(`/user/queue/train/${trainId}/comment`,
(subs) => {
  // 记住取消订阅函数, 界面销毁时取订
  this.unsubscribes.push(subs);
}, (data) => {
  data && this.pauseThenShowHint(data);
});

this.resultPromise = new Promise((resolve, reject) => {
  StompClient.subscribe(`/user/queue/train/${trainId}/result`,
  (subs) => {
    this.unsubscribes.push(subs);
  },
  (data) => {
    this.setState({ result: data }, ()=>{
      resolve(data);
    });
  }
  );
});
});
}

// 上报交易行为数据
uploadTradeBehavior = (barIndex, timestamp, action) => {

  const messageData = {
    barIndex: barIndex,
    actionTime: timestamp,
    actionSide: action,

```

```

    };
    StompClient.send(`/queue/train/${this.state.trainingInfo.trainId}/action`, messageData);
  }

  endPractice = () => {
    StompClient.send(`/queue/train/${this.state.trainingInfo.trainId}/end`);
  }

  // 倒计时开始
  countDownThenBegin = () => {
    // 开始倒计时
    this.remainingSeconds = this.state.countDownSeconds;
    this.silentCountDown = setInterval(() => {
      this.remainingSeconds--;
      if (this.remainingSeconds === 4) {
        UIDisplayingHelper.showToast("交易即将开始", 1.0);
      }
      if (this.remainingSeconds <= 3) {
        // 倒计时还剩3秒时收起K线图, 准备开始
        this.hideKlineModalView();
      }
    }, 1000);
  }

  // 开始交易
  beginTrading = () => {
    this.tick(); // 分时图开始走
    this.setState({ playing: true });
  }

  // 结束交易
  terminateTrading = () => {
    this.setState({ playing: false }, () => {
      const yieldRate = this.state.yieldRate;
      const id = this.getQueryParam('id');

      // 上报练习数据
      UIDisplayingHelper.showLoading();

      this.endPractice();

      TrainingService.addTrainingStat(id, yieldRate > 0).then(() => {
        UIDisplayingHelper.hideLoading();
        // 跳转到练习结果页面
        this.toResultView();
      }).catch((error) => {
        UIDisplayingHelper.hideLoading();
        console.error("保存练习数据时发生错误");
        // 跳转到练习结果页面
        this.toResultView();
      });
    });
  }

  // 继续
  resume = () => {
    this.setState({ playing: true }, () => {
      if (!(this.tickTimer)) {
        this.tick();
      }
    });
  }

  // 暂停
  pause = (callback) => {

```

```

    this.setState({ playing: false }, () => {
      this.tickTimer && clearTimeout(this.tickTimer);
      this.tickTimer = null;
      callback && callback();
    });
  }

  // 暂停然后弹提示框
  pauseThenShowHint = (hintText) => {
    this.pause(() => {
      TradingHintPopup.show(hintText, () => {
        // call back on dismiss
        this.resume();
      });
    });
  };

  // 跳转到结果页面
  toResultView = () => {
    const id = this.getQueryParam('id');
    UIDisplayingHelper.showLoading("正在计算成绩");

    this.resultPromise.then((result)=>{
      navigator.navigateToPracticeResultView(id, this.state.trainingInfo.trainId, true);
    })
  }

  // 交易分时图走动
  tick = () => {

    const { tradingActions, curTradeAction, curIndex, yieldRate } = this.state;
    const interval = (this.state.trainingInfo && this.state.trainingInfo.interval) || 300;

    const nextIndex = curIndex + 1;

    // 交易结束
    if (nextIndex >= this.timingData.barTotal) {
      this.terminateTrading();
      return false;
    }

    // 异常处理
    if (this.timingData.timestamps.length === 0) {
      console.warn('Timing list is not ready!');
      this.tickTimer = setTimeout(this.tick, interval);
      return false;
    }

    // 计算新收益
    const curBarItem = this.timingData.barList[nextIndex];
    let newYieldRate = yieldRate;
    if (curTradeAction !== Constants.TRADE_ACTIONS.CLOSE) {
      const { close } = curBarItem;
      newYieldRate = calYieldRate({
        actions: tradingActions,
        close,
        lever: 1,
      });
    }

    // 刷新分时图
    this.timingChart.updateChart(this.sliceTimingData());

    // 更新index、收益 etc
    this.setState({
      curIndex: nextIndex,
      curBarItem: curBarItem,

```

```

        yieldRate: newYieldRate
    });

    // 分时下一跳
    this.tickTimer = setTimeout(this.tick, interval);
}

// 截取当前时刻区间的交易数据
sliceTimingData = () => {

    let sliceEndIndex = this.state.curIndex >= this.timingData.timestamps.length-1 ?
    this.timingData.timestamps.length : this.state.curIndex+1;
    const currentTimingData = {
        preClosePrice: this.timingData.preClosePrice,
        curBarItem: this.state.curBarItem,
        tradingActions: this.state.tradingActions,
        timestamps: this.timingData.timestamps,
        prices: this.timingData.prices.slice(0, sliceEndIndex),
        averagePrices: this.timingData.averagePrices.slice(0, sliceEndIndex),
        volumes: this.timingData.volumes.slice(0, sliceEndIndex),
        volumesWithFlag: this.timingData.volumesWithFlag.slice(0, sliceEndIndex),
        macd: this.timingData.macd.slice(0, sliceEndIndex),
        dea: this.timingData.dea.slice(0, sliceEndIndex),
        diff: this.timingData.diff.slice(0, sliceEndIndex),
        klineData: this.timingData.klineData.slice(0, sliceEndIndex),
        chartStyle: this.state.timingChartStyle
    }
    // 更新交易量、最高、最低价
    this.setState({
        currentVolumes: currentTimingData.volumes.reduce((a, b) => a + b, 0),
        currentHighestPrice: Math.max.apply({}, currentTimingData.prices),
        currentLowestPrice: Math.min.apply({}, currentTimingData.prices),
    });
    return currentTimingData;
}

// 点击买入
handleBuyButtonClick = () => {
    this.handleTradeAction(Constants.TRADE_ACTIONS.BUY);
}

// 点击卖出
handleSellButtonClick = () => {
    this.handleTradeAction(Constants.TRADE_ACTIONS.SELL);
}

// 点击平仓
handleCloseButtonClick = () => {
    this.handleTradeAction(Constants.TRADE_ACTIONS.CLOSE);
}

// 处理交易动作
handleTradeAction = (actionSide) => {

    const { tradingActions, curIndex, curBarItem: { close, timestamp } } = this.state;

    // 所有历史交易行为（计算收益率等用）
    tradingActions.push({
        curIndex,
        timestamp,
        side: actionSide,
        close,
    });

    // 上报交易行为数据
    this.uploadTradeBehavior(curIndex, new Date().getTime(), actionSide);
}

```



```

// 更新当前交易仓位 (买仓, 卖仓, 平仓)
this.setState({
  curTradeAction: actionSide
});
}

// 点击历史K线按钮
handleKlineChartButtonClick = () => {
  this.showKlineModalView();
}

// 关闭K线图
handleKlineModalViewCloseButtonClick = () => {
  this.hideKlineModalView();
}

showKlineModalView = () => {
  this.setState({
    displayNoneKlineModalView: false
  }, () => {
    this.setState({ showKlineModalView: true });
  });
}

hideKlineModalView = () => {
  this.setState({
    showKlineModalView: false
  }, () => {
    setTimeout(() => {
      this.setState({ displayNoneKlineModalView: true });
    }, 300);
  });
}

if (this.silentCountDown) {
  this.silentCountDown && clearInterval(this.silentCountDown);
  this.silentCountDown = null;
  BounceCountDown.show(this.remainingSeconds, () => {
    this.beginTrading();
  });
}

// Render view
render() {
  const { curTradeAction, yieldRate, tradingActions } = this.state;
  const currentPrice = (this.state.curBarItem && this.state.curBarItem.close) || 0.00;
  const preClosePrice = (this.timingData && this.timingData.preClosePrice) || 0.00;
  const priceChange = currentPrice === 0 ? 0.00 : (currentPrice - preClosePrice);
  const priceChangeRatio = currentPrice === 0 ? 0.00 : ((currentPrice / preClosePrice - 1.0) * 100);
  // 是否已平仓, 控制交易按钮可用状态
  const isClosed = this.state.curTradeAction === Contants.TRADE_ACTIONS.CLOSE;

  const dayKline = this.state.trainingInfo && this.state.trainingInfo.puzzle.history.dayKline;

  return (
    <ViewContainer>

      <div className="practice-view-wrapper">

        {/* 头部信息 */}
        <HeaderBanner
          currentPrice={currentPrice}
          priceChange={priceChange}
          priceChangeRatio={priceChangeRatio}
          currentVolumes={this.state.currentVolumes}
          currentLowestPrice={this.state.currentLowestPrice}
          currentHighestPrice={this.state.currentHighestPrice}

```

```

        emptyText={"训练"}
        yieldRate={yieldRate}
        tradingActions={tradingActions}
        curTradeAction={curTradeAction} />
    { /* 走势图 */ }
    <div className="timing-chart-wrapper">
        { this.timingData && <TimingChart showMarkPoint showMarkLine showRangeChart ref={node =>
(this.timingChart = node)} /> }
    </div>
    { /* 底部信息 */ }
    <BottomBanner />

    { /* 底部交易按钮 */ }
    {
        this.state.trainingInfo &&
        <ActionButtonPanel>
            <ActionButtonPanel.Button disabled={! (this.state.playing) || !isClosed}
type={ActionButtonPanel.Button.types.BUY} weight={3} onClick={this.handleBuyButtonClick}>
                买入
            </ActionButtonPanel.Button>
            <ActionButtonPanel.Button disabled={! (this.state.playing) || isClosed}
type={ActionButtonPanel.Button.types.CLOSE} weight={3} onClick={this.handleCloseButtonClick}>
                平仓
            </ActionButtonPanel.Button>
            { /* <ActionButtonPanel.Line />
            <ActionButtonPanel.Button disabled={! (this.state.playing)}
type={ActionButtonPanel.Button.types.HOLLOW} weight={1} onClick={() =>
{this.setState({timingChartStyle: (this.state.timingChartStyle=== 'kline'? 'timing': 'kline')}});} >
                {this.state.timingChartStyle=== 'kline'? '线': 'K'}
            </ActionButtonPanel.Button>
            <ActionButtonPanel.Button disabled={! (this.state.playing)}
type={ActionButtonPanel.Button.types.HOLLOW} weight={1} onClick={this.handleKlineChartButtonClick} >
                历史
            </ActionButtonPanel.Button> */ }
            </ActionButtonPanel>
        }

        {
            this.state.trainingInfo &&
            <ActionButtonPanel className="side-button-wrapper">
                <ActionButtonPanel.Button disabled={! (this.state.playing)}
type={ActionButtonPanel.Button.types.HOLLOW} weight={1} onClick={() =>
{this.setState({timingChartStyle: (this.state.timingChartStyle=== 'kline'? 'timing': 'kline')}});} >
                    {this.state.timingChartStyle=== 'kline'? <ChartLine />: 'K'}
                </ActionButtonPanel.Button>
                <ActionButtonPanel.Button disabled={! (this.state.playing)}
type={ActionButtonPanel.Button.types.HOLLOW} weight={1} onClick={this.handleKlineChartButtonClick} >
                    <ChartBar />
                </ActionButtonPanel.Button>
            </ActionButtonPanel>
        }

        { /* K线面板，可收起隐藏 */ }
        {
            this.state.trainingInfo && <KlineChartModalView show={this.state.showKlineModalView}
                                                                historyData={dayKline}
                                                                fromDate={dayKline.timestamp[0]}

toDate={dayKline.timestamp[dayKline.timestamp.length - 1]}

onClose={this.handleKlineModalViewCloseButtonClick}

style={{display: (this.state.displayNoneKlineModalView? "none": "block")}} />
        }

        { /* 离开提示 */ }
        <Prompt when={this.state.playing} message="_你确定要退出练习吗?" />

```

```

        </div>

    </ViewContainer>
  );
}
}

export default PracticeView;

import React, { Component } from 'react';
import MainArea from './MainArea';
import './App.scss';
import NavBar from './NavBar';
import BackHomeOverlapBar from '../component/BackHomeOverlapBar';
import WechatShareLayer from './WechatShareLayer';
import enableLocationChangeHook from '../component/hoc/enableLocationChangeHook';
import browserHistory from '../routes/history';
import ShareHelper from '../utils/ShareHelper';
import navigator from '../routes/navigator';
import CommonUtil from '../utils/CommonUtil';
import WechatHelper from '../utils/WechatHelper';
import RuntimeEnv from '../utils/RuntimeEnv';

class App extends Component {

  state = {
    navBarCanGoBack: false,
    showShareLayer: false
  }

  componentWillMount() {
    this.injectCommonMethods();
  }

  injectCommonMethods = () => {
    Component.prototype.setTitle = (title) => {
      if (RuntimeEnv.isInWechat()) {
        WechatHelper.setWechatNavBarTitle('小Q教练');
        // WechatHelper.setWechatNavBarTitle(title);
      }
      this.navBar && this.navBar.setTitle(title);
    };
    Component.prototype.setShowNavBarMoreButton = (show = false) => {
      this.navBar && this.navBar.setShowMoreButton(show);
    };
    Component.prototype.setNavBarMoreButtonTitle = (title) => {
      this.navBar && this.navBar.setMoreButtonTitle(title);
    };
    Component.prototype.setNavBarMoreButtonHandler = (handler) => {
      this.navBar && this.navBar.setMoreButtonHandler(handler);
    };
    Component.prototype.showNavBar = (show = false) => {
      // if (!(RuntimeEnv.isInWechat())) {
      this.navBar && this.navBar.show(show);
      // }
    };
    Component.prototype.hideBackHomeOverlap = () => {
      this.backHomeOverlapBar && this.backHomeOverlapBar.hide();
    };
    Component.prototype.showShareLayer = () => {
      this.setState({ showShareLayer: true });
    };
  }

  hideShareLayer = () => {
    this.setState({ showShareLayer: false });
  }
}

```

```

handleLocationChange = (location) => {
  // init wechat jssdk
  ShareHelper.initWxJSSDK();
  ShareHelper.resetDefaultShareConfig();
  // UIDisplayingHelper.hideLoading();
  this.updateBackHomeOverlapBarVisibility(location);
  this.setState({
    navBarCanGoBack: (location.pathname !== '/home' && (CommonUtil.isOpenedFromShare(location) ||
    browserHistory.length > 1)),
  });
}

handleNavBarBackButtonClick = () => {
  if (CommonUtil.isOpenedFromShare(window.location.href)) {
    navigator.navigateToHomeView();
  } else {
    browserHistory.goBack();
  }
}

handleBackHomeOverlapBarClick = () => {
  navigator.navigateToHomeView();
}

updateBackHomeOverlapBarVisibility = (location) => {
  // let show = CommonUtil.isOpenedFromShare(location);
  let show = false; // 改用左上角回退按钮点击返回, 返回Home浮层总是隐藏
  if (show) {
    this.backHomeOverlapBar && this.backHomeOverlapBar.show();
  } else {
    this.backHomeOverlapBar && this.backHomeOverlapBar.hide();
  }
}

render() {
  return (
    <div className="app-wrapper">
      <NavBar canGoBack={this.state.navBarCanGoBack}
        handleGoBack={this.handleNavBarBackButtonClick}
        ref={(node) => this.navBar = node} />
      <MainArea />
      <BackHomeOverlapBar ref={(node) => this.backHomeOverlapBar = node}
        onClick={this.handleBackHomeOverlapBarClick} />
      { this.state.showShareLayer && <WechatShareLayer onClick={this.hideShareLayer} /> }
    </div>
  );
}
}

export default enableLocationChangeHook(App);

```

```

import React, { Component } from 'react';
import ViewContainer from '../skeleton/ViewContainer';
import RankingService from '../service/RankingService';
import UIDisplayingHelper from '../component/helper/UIDisplayingHelper';
import navigator from '../routes/navigator';
import TabBar from './TabBar';
import RankingList from './RankingList';
import ShareHelper from '../utils/ShareHelper';
import './RankingListView.scss';

const SCORE_TAB = 0;
const BALANCE_TAB = 1;

class RankingListView extends Component {

```

```

state = {
  tabIndex: SCORE_TAB,
  balanceRankingList: null,
  scoreRankingList: null
}

componentDidMount() {
  this.showNavBar(true);
  this.setTitle("排行榜");
  this.loadTabList();
}

// 设置当前页面的分享信息
setShareInfo = () => {
  setTimeout(() => {
    ShareHelper.setShareTitle('小T排行榜');
    ShareHelper.setShareDesc('A股玩得很6？一局比赛打回原型，赶紧戳进来试试吧！');
  }, 0);
}

loadTabList = (ignoreCache = false) => {
  if (this.state.tabIndex === SCORE_TAB) {
    if (this.state.scoreRankingList != null && !ignoreCache) return;
    this.loadScoreRankingList();
  } else if (this.state.tabIndex === BALANCE_TAB) {
    if (this.state.balanceRankingList != null && !ignoreCache) return;
    this.loadBalanceRankingList();
  }
}

loadBalanceRankingList = () => {
  UIDisplayingHelper.showLoading();
  RankingService.getRichList().then((data) => {
    UIDisplayingHelper.hideLoading();
    this.setState({balanceRankingList: data});
  });
}

loadScoreRankingList = () => {
  UIDisplayingHelper.showLoading();
  RankingService.getScoreList().then((data) => {
    UIDisplayingHelper.hideLoading();
    this.setState({scoreRankingList: data});
    this.setShareInfo();
  });
}

handleTabClick = (tabIndex) => {
  this.setState({
    tabIndex: tabIndex
  }, () => {
    this.loadTabList();
  });
}

handleListCellClick = (index) => {
  let targetUser = null;
  if (this.state.tabIndex === SCORE_TAB) {
    targetUser = this.state.scoreRankingList[index];
  } else if (this.state.tabIndex === BALANCE_TAB) {
    targetUser = this.state.balanceRankingList[index];
  }
  navigator.navigateToProfileView(targetUser);
}

render() {
  return (

```

```

<ViewContainer>
  { /* Tab Bar */ }
  <TabBar>
    <TabBar.Item selected={this.state.tabIndex===SCORE_TAB}
      onClick={() => this.handleTabClick(SCORE_TAB)}>
      Score排行榜
    </TabBar.Item>
    <TabBar.Item selected={this.state.tabIndex===BALANCE_TAB}
      onClick={() => this.handleTabClick(BALANCE_TAB)}>
      财富排行榜
    </TabBar.Item>
  </TabBar>
  { /* Score Ranking List */ }
  <RankingList show={this.state.tabIndex===SCORE_TAB}>
    {
      this.state.scoreRankingList && this.state.scoreRankingList.map((user, index) => {
        return <RankingList.Cell key={user.userId}
          user={user}
          number={index+1}
          rankingKey="fdtscore"
          onClick={() => this.handleListCellClick(index)} />
      })
    }
  </RankingList>
  { /* Balance Ranking List */ }
  <RankingList show={this.state.tabIndex===BALANCE_TAB}>
    {
      this.state.balanceRankingList && this.state.balanceRankingList.map((user, index) => {
        return <RankingList.Cell key={user.userId}
          user={user}
          number={index+1}
          rankingKey="money"
          onClick={() => this.handleListCellClick(index)} />
      })
    }
  </RankingList>
</ViewContainer>
);
}
}

```

```
export default RankingListView;
```

```

import React, { Component } from 'react';
import ViewContainer from '../../skeleton/ViewContainer';
import UIDisplayingHelper from '../../component/helper/UIDisplayingHelper';
import TrainingService from '../../service/TrainingService';
import ActionButtonPanel from '../../component/ActionButtonPanel';
import navigator from '../../routes/navigator';
import CommonUtil from '../../utils/CommonUtil';
import { renderKlineChart } from '../PortalView/KlineChartRender';
import { renderMacdChart } from '../PortalView/MACDChartRender';
import { renderMeanReversionChart } from '../PortalView/MeanReversionChartRender';
import { renderBBandChart } from '../PortalView/BBandChartRender';
import { renderRsiChart } from '../PortalView/RSIChartRender';
import './TrainingItemPresentationView.scss';

```

```
class TrainingItemPresentationView extends Component {
```

```

  state = {
    details: null
  }

```

```

  componentDidMount() {
    this.showNavBar(true);
  }

```

```

    if (this.validateUrlParams()) {
      this.setTitle(this.isPatternTraining()?"K线形态":"技术指标");
      this.isPatternTraining() ? this.getPatternDetails() : this.getIndicatorDetails();
    }
  }

  isOpenedFrom = () => {
    if (this.getQueryParam('from')) {
      return true;
    }
    return false;
  }

  // 获取url传参
  getQueryParam = (paramKey) => {
    let queryParams = CommonUtil.parseParams(this.props.location.search);
    return queryParams[paramKey] || '';
  }

  validateUrlParams = () => {
    const { category, id } = this.props.match.params;
    if (!category) {
      UIDisplayingHelper.showToast("需要传入类别参数");
      return false;
    }
    else if (!id) {
      UIDisplayingHelper.showToast("需要传入ID");
      return false;
    }
    return true;
  }

  isPatternTraining = () => {
    return this.props.match.params.category === "patterns";
  }

  // 获取形态数据
  getPatternDetails = () => {
    const { id } = this.props.match.params;
    UIDisplayingHelper.showLoading();
    TrainingService.getPatternDetails(id).then((detailData) => {
      UIDisplayingHelper.hideLoading();
      this.setState({
        details: detailData
      }, () => {
        this.renderPatternChart();
      });
    }).catch((e) => {
      UIDisplayingHelper.hideLoading();
      UIDisplayingHelper.showToast("获取形态详情数据时发生错误");
    });
  }

  renderPatternChart = () => {
    const renderData = {
      category: this.state.details.sample.bars.timestamp,
      klineData: this.state.details.sample.bars.ohlcv.map(bar => [bar.open, bar.close, bar.low, bar.high]),
      customizedRectBox: [{
        startIndex: this.state.details.sample.startIndex,
        endIndex: this.state.details.sample.endIndex,
        color: '#00E2FF'
      }]
    };
    renderKlineChart(this.chartBoard, renderData);
  }

  // 获取指标数据
  getIndicatorDetails = () => {

```

```

const { id } = this.props.match.params;
UIDisplayingHelper.showLoading();
TrainingService.getIndicatorDetails(id).then((detailData) => {
  UIDisplayingHelper.hideLoading();
  this.setState({
    details: detailData
  }, () => {
    this.renderIndicatorChart();
  });
}).catch((e) => {
  UIDisplayingHelper.hideLoading();
  UIDisplayingHelper.showToast("获取指标详情数据时发生错误");
});
}

renderIndicatorChart = () => {
  const indicatorId = this.state.details.indicatorId;
  // MACD
  if (indicatorId.toUpperCase().indexOf("MACD") >= 0) {
    const renderData = {
      category: this.state.details.sample.bars.timestamp,
      macd: this.state.details.sample.bars.macd,
      dea: this.state.details.sample.bars.dea,
      diff: this.state.details.sample.bars.diff,
      signalIndex: this.state.details.sample.signalIndex,
      signalMarkPointText: indicatorId.toUpperCase().indexOf("UP") ? "B" : "S"
    };
    renderMacdChart(this.chartBoard, renderData);
  }
  // BBand (布林带)
  else if (indicatorId.toUpperCase().indexOf("BBAND") >= 0) {
    const renderData = {
      category: this.state.details.sample.bars.timestamp,
      klineData: this.state.details.sample.bars.ohlc.map(bar => [bar.open, bar.close, bar.low,
bar.high]),
      bbandsUp: this.state.details.sample.bars.bbands_up,
      bbandsMiddle: this.state.details.sample.bars.bbands_middle,
      bbandsLow: this.state.details.sample.bars.bbands_low,
      signalIndex: this.state.details.sample.signalIndex,
      signalMarkPointText: indicatorId.toUpperCase().indexOf("UP") ? "B" : "S"
    };
    renderBBandChart(this.chartBoard, renderData);
  }
  // MeanReversion (sma line cross)
  else if (indicatorId.toUpperCase().indexOf("MEANREVERSION") >= 0) {
    const renderData = {
      category: this.state.details.sample.bars.timestamp,
      klineData: this.state.details.sample.bars.ohlc.map(bar => [bar.open, bar.close, bar.low,
bar.high]),
      ma10: this.state.details.sample.bars.ma10,
      ma30: this.state.details.sample.bars.ma30,
      signalIndex: this.state.details.sample.signalIndex,
      signalMarkPointText: indicatorId.toUpperCase().indexOf("UP") ? "B" : "S"
    };
    renderMeanReversionChart(this.chartBoard, renderData);
  }
  // RSI
  else if (indicatorId.toUpperCase().indexOf("RSI") >= 0) {
    const renderData = {
      category: this.state.details.sample.bars.timestamp,
      rsi6: this.state.details.sample.bars.rsi6,
      rsi14: this.state.details.sample.bars.rsi14,
      rsi24: this.state.details.sample.bars.rsi24,
      signalIndex: this.state.details.sample.signalIndex,
      signalMarkPointText: indicatorId.toUpperCase().indexOf("UP") ? "B" : "S"
    };
    renderRsiChart(this.chartBoard, renderData);
  }
}

```



```

}

handlePracticeAgainButtonClick = (replace) => {
  navigator.navigateToTechTrainingPracticeView(this.props.match.params.category,
  this.props.match.params.id, replace);
}

// UI绘制组件拆分
renderSplitting = {
  // 页面视图容器
  ContentWrapper: ({children}) => (
    <div className="training-item-presentation-view-wrapper">
      {children}
    </div>
  ),
  // 标题
  Title: ({children}) => (
    <div className="title">
      {children}
    </div>
  ),
  // 示例图
  SampleKlineChart: () => (
    <div className="chart-wrapper">
      <div className="title-background"></div>
      <div className="title">{this.state.details.cnName}</div>
      <div className="chart" ref={node => (this.chartBoard = node)}>/>
    </div>
  ),
  // 描述
  Description: ({children}) => (
    <div className="desc">
      {children}
    </div>
  ),
}

render() {
  const ContentWrapper = this.renderSplitting.ContentWrapper;
  const Title = this.renderSplitting.Title;
  const SampleKlineChart = this.renderSplitting.SampleKlineChart;
  const Description = this.renderSplitting.Description;

  return (
    <ViewContainer>
      {
        this.state.details &&
        <ContentWrapper>
          {!this.isOpenedFrom() && <Title>知识点回顾</Title>}
          <SampleKlineChart />
          <Description>{this.state.details.desc}</Description>
        </ContentWrapper>
      }
      {
        this.state.details &&
        (this.isOpenedFrom()
          ? <ActionButtonPanel style={{background: 'transparent', position: 'absolute'}}>
            <ActionButtonPanel.Button type={ActionButtonPanel.Button.types.HOLLOW} weight={1}
            onClick={() => navigator.navigateToTrainingView(false, this.props.match.params.category)}>
              {this.isPatternTraining()? "其他形态": "其他指标"}
            </ActionButtonPanel.Button>
            <ActionButtonPanel.Button weight={2} onClick={() =>
              this.handlePracticeAgainButtonClick(false)}>
              开始练习
            </ActionButtonPanel.Button>
          </ActionButtonPanel>
        )
      }
    </ViewContainer>
  );
}

```

```

      : <ActionButtonPanel style={{background:'transparent',position:'absolute'}}>
        <ActionButtonPanel.Button type={ActionButtonPanel.Button.types.HOLLOW} onClick={() =>
this.handlePracticeAgainButtonClick(true)}>
          重新练习
        </ActionButtonPanel.Button>
        <ActionButtonPanel.Button onClick={() => navigator.navigateToTrainingView(true,
this.props.match.params.category)}>
          更多{this.isPatternTraining()?"K线形态":"指标训练"}
        </ActionButtonPanel.Button>
      </ActionButtonPanel>
    }
  </ViewContainer>
);
}
}
}

export default TrainingItemPresentationView;

import React, { Component } from 'react';
import echarts from '../../charts/echarts';
import chartColors from '../../charts/chartColors';
import CommonUtil from '../../utils/CommonUtil';
import ActionButtonPanel from '../../component/ActionButtonPanel';
import './GuessKlinePracticeChart.scss';

const px = number => number * CommonUtil.getDPR();

class GuessKlinePracticeChart extends Component {

  state = {
    selected: false,
    selectedDirection: "unknown"
  }

  componentDidMount() {
    this.chart = echarts.init(this.chartBoard, null, {renderer:'canvas'});
    this.chart.resize();
    this.updateChart();
  }

  componentWillUnmount() {
    this.chart && this.chart.dispose();
  }

  updateChart = () => {

    const { puzzleData } = this.props;
    const splitInterval = 10;

    let klineLength = puzzleData.bars.ohlc.length >= 15 ? 15 : puzzleData.bars.ohlc.length;
    const categoryData = Array(klineLength*2).fill(0);
    const klineData = [];
    let startPoint = 0;
    if (puzzleData.endIndex >= klineLength) {
      startPoint = puzzleData.endIndex - (klineLength - 1);
    }
    for (let i = startPoint, j = 0; i < (startPoint+klineLength+(this.state.selected?1:0)); i++, j++) {
      categoryData[j] = puzzleData.bars.timestamp[i];
      if (i < puzzleData.bars.ohlc.length) {
        klineData[j] = [puzzleData.bars.ohlc[i].open, puzzleData.bars.ohlc[i].close,
puzzleData.bars.ohlc[i].low, puzzleData.bars.ohlc[i].high];
      }
    }

    let option = {
      backgroundColor: 'transparent',
      animation: true,
      grid: [

```

```

{
  show: true,
  borderWidth: px(1),
  borderColor: chartColors.splitLineColor,
  top: '2%',
  left: '50%',
  right: '2%',
  bottom: '2%'
}
],
xAxis: [
  {
    type: 'category',
    gridIndex: 0,
    data: categoryData,
    boundaryGap: true,
    axisLine: {show: false},
    axisTick: {show: false},
    axisLabel: {show: false},
    splitLine: {
      show: true,
      lineStyle: {
        color: chartColors.splitLineColor
      },
      interval: index => {
        return index % splitInterval === 0;
      }
    }
  }
],
yAxis: [
  {
    z: 10,
    scale: true,
    gridIndex: 0,
    axisTick: {show: false},
    axisLine: {show: false},
    axisLabel: {
      show: true,
      inside: false,
      margin: 10,
      showMinLabel: true,
      showMaxLabel: true,
      padding: [10, 0],
      verticalAlign: 'center',
      color: chartColors.labelColor,
      fontSize: px(10)
    },
    splitLine: {
      show: true,
      lineStyle: {
        color: chartColors.splitLineColor
      }
    },
    splitNumber: 3,
    boundaryGap: ['10%', '25%']
  }
],
series: [
  {
    name: 'Kline',
    type: 'candlestick',
    silent: true,
    data: klineData,
    itemStyle: {
      normal: {
        color: chartColors.upColor,
        color0: chartColors.downColor,
        borderColor: null,

```

```

        borderColor0: null
      }
    }
  ]
};

// 还未选择, 显示下一交易日灰色bar ?, 提示猜测涨跌
if (!this.state.selected) {
  let preKlineBar = klineData[klineData.length-1];
  let questionMark = {
    type: 'custom',
    name: 'questionMark',
    z: 100,
    data: [[klineLength, preKlineBar[2] + (preKlineBar[3]-preKlineBar[2])/2]],
    renderItem: function (params, api) {

      const index = api.value(0); // index
      const value = api.value(1); // y position value
      const position = api.coord([index, value]); // center position
      const color = '#0088FF';
      const actionText = "?";
      const rectSize = px(20), gap = px(5), lineLength = px(30);
      const lineEndPoint = {x: 0, y: -(lineLength)}; // 线尾的位置
      const textPoint = {x: lineEndPoint.x, y: lineEndPoint.y - gap - rectSize / 2}; // 文字绘制位置
      const opacity = 0.8;

      return {
        type: 'group',
        position: [position[0], position[1]],
        children: [
          {
            type: 'rect',
            shape: {
              x: px(-10/2),
              y: px(-60/2),
              width: px(10),
              height: px(60)
            },
            style: {
              fill: '#999999',
              opacity: 0.6
            },
          },
          {
            type: 'group',
            position: [0, -px(60/2+10)],
            children: [
              {
                type: 'polyline',
                shape: {
                  points: [[0, -px(4)], [lineEndPoint.x, lineEndPoint.y]]
                },
                style: {
                  lineWidth: px(1),
                  stroke: '#00E2FF',
                  opacity: 0.5
                },
              },
              {
                type: 'circle',
                style: {
                  opacity: 0.5,
                  fill: '#00E2FF',
                },
                shape: {
                  cx: 0,
                  cy: 0,

```

```

        r: px(4)
      },
    },
    {
      type: 'rect',
      shape: {
        x: lineEndPoint.x - rectSize / 2,
        y: lineEndPoint.y - gap - rectSize,
        width: rectSize,
        height: rectSize
      },
      style: {
        fill: color,
        opacity: opacity
      },
    },
    {
      type: 'polygon',
      shape: {
        points: [[lineEndPoint.x, lineEndPoint.y - gap + 5], [lineEndPoint.x - 6,
lineEndPoint.y - gap], [lineEndPoint.x + 6, lineEndPoint.y - gap]]
      },
      style: {
        fill: color,
        opacity: opacity
      },
    },
    {
      type: 'text',
      z: 100,
      zIndex: 100,
      style: {
        opacity: opacity,
        fill: 'white',
        text: actionText,
        font: `${px(14)}px "Open Sans", sans-serif`,
        x: textPoint.x,
        y: textPoint.y,
        textAlign: 'center',
        textVerticalAlign: 'middle'
      }
    }
  ]
}
];
};
}
}
option.series.push(questionMark);
}

this.chart.setOption(option, {
  notMerge: true
});
}

getPracticeResult = () => {
  return this.state.selectedDirection === this.props.puzzleData.answer;
}

handleButtonClick = (direction) => {
  this.setState({
    selected: true,
    selectedDirection: direction
  }, () => {
    this.updateChart();
    this.props.onFinished && this.props.onFinished(this.getPracticeResult());
  });
}
}

```

```

render() {
  return (
    <div className="guess-kline-practice-chart-wrapper">
      <div className="chart" ref={node => (this.chartBoard = node)} />
      <ActionButtonPanel style={{left: '0.4rem', background: 'transparent', position: 'absolute'}}>
        <ActionButtonPanel.Button disabled={this.state.selected}
type={ActionButtonPanel.Button.types.BUY} onClick={() => this.handleButtonClick('up')}>看涨
      </ActionButtonPanel.Button>
        <ActionButtonPanel.Button disabled={this.state.selected}
type={ActionButtonPanel.Button.types.SELL} onClick={() => this.handleButtonClick('down')}>看跌
      </ActionButtonPanel.Button>
      </ActionButtonPanel>
    </div>
  );
}
}

```

```
export { GuessKlinePracticeChart };
```

```

import React, { Component } from 'react';
import PropTypes from 'prop-types';
import { renderKlineChart } from '../PortalView/KlineChartRender';
import { renderMacdChart } from '../PortalView/MACDChartRender';
import { renderMeanReversionChart } from '../PortalView/MeanReversionChartRender';
import { renderBBandChart } from '../PortalView/BBandChartRender';
import { renderRsiChart } from '../PortalView/RSIChartRender';
import './ChoiceOption.scss';

class ChoiceOption extends Component {

  componentDidMount() {
    let { category, id, optionData } = this.props;
    if (category === "patterns") {
      renderKlineChart(this.chartBoard, {
        category: Array(Math.max(optionData.ohlc.length-1, 10)).fill(0),
        klineData: optionData.ohlc.map(bar => [bar.open, bar.close, bar.low, bar.high])
      });
    }
    else {
      // MACD
      if (id.toUpperCase().indexOf("MACD") >= 0) {
        const renderData = {
          category: optionData.timestamp,
          macd: optionData.macd,
          dea: optionData.dea,
          diff: optionData.diff
        };
        renderMacdChart(this.chartBoard, renderData);
      }
      // BBand (布林带)
      else if (id.toUpperCase().indexOf("BBAND") >= 0) {
        const renderData = {
          category: optionData.timestamp,
          klineData: optionData.ohlc.map(bar => [bar.open, bar.close, bar.low, bar.high]),
          bbandsUp: optionData.bbands_up,
          bbandsMiddle: optionData.bbands_middle,
          bbandsLow: optionData.bbands_low
        };
        renderBBandChart(this.chartBoard, renderData);
      }
      // MeanReversion (sma line cross)
      else if (id.toUpperCase().indexOf("MEANREVERSION") >= 0) {
        const renderData = {
          category: optionData.timestamp,
          klineData: optionData.ohlc.map(bar => [bar.open, bar.close, bar.low, bar.high]),

```

```

        ma10: optionData.ma10,
        ma30: optionData.ma30
    });
    renderMeanReversionChart(this.chartBoard, renderData);
}
// RSI
else if (id.toUpperCase().indexOf("RSI") >= 0) {
    const renderData = {
        category: optionData.timestamp,
        rsi6: optionData.rsi6,
        rsi14: optionData.rsi14,
        rsi24: optionData.rsi24
    };
    renderRsiChart(this.chartBoard, renderData);
}
}
}

render() {
    return (
        <div className={`pattern-practice-choice-option-wrapper ${this.props.highlight?'highlight':''}`}
            style={this.props.reveal ? {
                opacity: (this.props.highlight ? 1.0 : 0.7),
                borderColor: (this.props.highlight ?
(this.props.index===this.props.correctIndex?'#00da3c':'red') : '#0088FF')
            } : {}}
            onClick={this.props.onClick}>
            <div className="item-seq-no">[['A', 'B', 'C'][this.props.index]]</div>
            <div className="chart" ref={node => (this.chartBoard = node)}>
                {this.props.reveal && <img alt="" className="answer-indicator" src={this.props.index ===
this.props.correctIndex
?
require('../../../../images/training/correct_icon.jpg')
:
require('../../../../images/training/wrong_icon.jpg')} />}
            </div>
        );
    };
}

ChoiceOption.propTypes = {
    reveal: PropTypes.bool,
    highlight: PropTypes.bool,
    correctIndex: PropTypes.number,
    index: PropTypes.number,
    onClick: PropTypes.func
}

ChoiceOption.defaultProps = {
    reveal: false,
    highlight: false,
    correctIndex: -1,
    index: 0,
    onClick: null
}

export { ChoiceOption };

import React, { Component } from 'react';
import PropTypes from 'prop-types';

class RankingListCell extends Component {
    render() {
        return (
            <div className="ranking-list-cell-wrapper" onClick={() => (this.props.onClick &&
this.props.onClick())}>
                <div className="number">{this.props.number}</div>
            </div>
        );
    }
}

```

```

    <div className="avatar-wrapper">
      <img className="avatar-img" alt="" src={this.props.user.headimgurl} />
    </div>
    <div className="name-info">
      <div className="username">{this.props.user.nickname}</div>
      <div className="organization">{this.props.user.organization}</div>
    </div>
    <div className="score">
      `{this.props.rankingKey==='money'?"¥":""}${this.props.user[this.props.rankingKey]}`
    </div>
    <div className="ranking-list-row-bottom-line" />
  </div>
  );
}
}

RankingListCell.propTypes = {
  onClick: PropTypes.func,
  number: PropTypes.number.isRequired,
  user: PropTypes.object.isRequired,
  rankingKey: PropTypes.string.isRequired
}

class RankingList extends Component {
  render() {
    return (
      <div className="ranking-list-wrapper" style={{display:(this.props.show?"block":"none")}}>
        {this.props.children}
      </div>
    );
  }
}

RankingList.propTypes = {
  show: PropTypes.bool
}

RankingList.Cell = RankingListCell;

export default RankingList;

import React, { Component } from 'react';
import PropTypes from 'prop-types';

class TabItem extends Component {
  render() {
    return (
      <div className="tab-item" onClick={() => (this.props.onClick && this.props.onClick())}>
        <div className="title">{this.props.children}</div>
        <div className="indicator" style={{visibility:(this.props.selected?"visible":"hidden")}} />
      </div>
    );
  }
}

TabItem.propTypes = {
  onClick: PropTypes.func,
  selected: PropTypes.bool,
}

class Tabs extends Component {
  render() {
    return (
      <div className="ranking-list-tabs-wrapper">
        {this.props.children}
        <div className="ranking-list-row-bottom-line" />
      </div>
    );
  }
}

```



```

    });
  }
}

Tabs.Item = TabItem;

export default Tabs;

import React, { Component } from 'react';
import SimilarKlineChart from '../SimilarKlineView/SimilarKlineChart';
import CommonUtil from '../../utils/CommonUtil';

class KlinePanel extends Component {

  render() {
    let similarWindowPercentage =
Math.ceil((this.props.sourceStockKlineData.timestamp.length/this.props.similarStockKlineData.timestamp.length)*100);
    return (
      <div className="comparing-kline-panel-wrapper">
        <div className="comparing-panel">
          /* 上半部K线 */
          <div className="kline-container">
            /* K线区间和股票名称 */
            <div className="range-text-wrapper top" style={{width:`${similarWindowPercentage}%`}}>
              <div className="text left">{this.props.sourceStockName}</div>
            </div>
            <div className="range-text-wrapper bottom" style={{width:`${similarWindowPercentage}%`}}>
              <div className="text
left">{CommonUtil.getFormatDateString(this.props.sourceStockKlineData.timestamp[0])}</div>
              {/*<div className="text right"
style={{width:"37%"}}>{CommonUtil.getFormatDateString(this.props.sourceStockKlineData.timestamp[Math.max(0
, this.props.sourceStockKlineData.timestamp.length-1)])}</div>*/}
            </div>
            /* K线(上) */
            <SimilarKlineChart range={this.props.similarStockKlineData.timestamp.length}
              showTillBarIndex={this.props.sourceStockKlineData.timestamp.length}
              klineData={this.props.sourceStockKlineData} />

            /* 右侧提示 */
            <div className="right-side-mask center-text" style={{width:`${100-
similarWindowPercentage}%`}}>
              未来趋势?
            </div>
          </div>
          /* 下半部K线 */
          <div className="kline-container">
            /* K线区间和股票名称 */
            <div className="range-text-wrapper top" style={{width:`${similarWindowPercentage}%`}}>
              <div className="text left">{this.props.similarStockName}</div>
            </div>
            <div className="range-text-wrapper bottom" style={{width:`${similarWindowPercentage}%`}}>
              <div className="text
left">{CommonUtil.getFormatDateString(this.props.similarStockKlineData.timestamp[0])}</div>
              {/*<div className="text right"
style={{width:"37%"}}>{CommonUtil.getFormatDateString(this.props.sourceStockKlineData.timestamp[Math.max(0
, this.props.sourceStockKlineData.timestamp.length-1)])}</div>*/}
            </div>
            /* K线(下) */
            <SimilarKlineChart range={this.props.similarStockKlineData.timestamp.length}
              showTillBarIndex={this.props.similarStockKlineData.timestamp.length}
              klineData={this.props.similarStockKlineData}
              actions={this.props.actions} />

          </div>
          <div className="highlight-box" style={{width:`${similarWindowPercentage}%`}} />
        </div>
      </div>
    );
  }
}

```

```

    });
  }
}

export default KlinePanel;

import React, { Component } from 'react';
import ViewContainer from '../../skeleton/ViewContainer';
import KlinePatternPracticeChart from '../../charts/KlinePatternPracticeChart';
import ActionButtonPanel from '../../component/ActionButtonPanel';
import UIDisplayingHelper from '../../component/helper/UIDisplayingHelper';
import Contants from '../../common/Contants';
import navigator from '../../routes/navigator';
import './SimilarKlinePracticeView.scss';

class SimilarKlinePracticeView extends Component {

  state = {
    myStockHistoryData: undefined,
    historyData: undefined,
    preYieldRate: 0.0,
    currentYieldRate: 0.0,
    empty: true,
    buyPrice: 0.0,
    tradeEnd: false
  }

  componentDidMount() {
    this.setTitle("相似K线复盘练习");
    this.showNavBar(true);
    if (!this.props.location.state) {
      UIDisplayingHelper.showToast("无题目数据");
    } else {
      this.setState({
        myStockHistoryData: this.props.location.state.sourceKlineData,
        historyData: this.props.location.state.similarKlineData.dayKlineVO
      }, () => {
        this.nextTick();
      });
    }
  }

  // 点击买入
  handleBuyButtonClick = () => {
    this.setState({
      empty: false,
      buyPrice: this.state.historyData.oclh[this.currentBarIndex][1]
    }, () => {
      this.addTradeAction(this.currentBarIndex, Contants.TRADE_ACTIONS.BUY);
      this.nextTick();
    });
  }

  // 点击卖出
  handleSellButtonClick = () => {
    let newYieldRate = this.calculateCurrentYieldRate()
    this.setState({
      empty: true,
      preYieldRate: newYieldRate
    }, () => {
      this.addTradeAction(this.currentBarIndex, Contants.TRADE_ACTIONS.SELL);
      this.nextTick();
    });
  }

  // 交易行为

```

```

addTradeAction = (index, side) => {
  if (index < this.state.historyData.timestamp.length) {
    this.actions.push({
      index: index,
      timestamp: this.state.historyData.timestamp[index],
      close: this.state.historyData.oclh[index][1],
      side: side
    });
  }
}
// 点击观望
handleNoTradeButtonClick = () => {
  this.nextTick();
}
// 下一根K线
currentBarIndex = !this.props.location.state ? 0 :
(this.props.location.state.sourceKlineData.timestamp.length-1)
actions = []
nextTick = () => {
  if (this.currentBarIndex < this.state.historyData.timestamp.length-1) {
    this.currentBarIndex++;
    this.klineChart.slideToIndex(this.currentBarIndex, this.actions);
    this.setState({
      currentYieldRate: this.calculateCurrentYieldRate()
    });
  } else {
    this.showPracticeResult();
  }
}
// 计算收益
calculateCurrentYieldRate = () => {
  let preYieldRate = this.state.preYieldRate;
  if (this.state.empty === false) {
    let currentPrice = this.state.historyData.oclh[this.currentBarIndex][1];
    let yieldRate = preYieldRate + ((currentPrice / this.state.buyPrice) - 1.0);
    return yieldRate;
  } else {
    return preYieldRate;
  }
}
// 复盘练习结束
showPracticeResult = () => {
  this.setState({
    tradeEnd: true
  });
  // UIDisplayingHelper.showDialog("提示", `复盘练习已结束, 您的收益是${(this.calculateCurrentYieldRate() *
100).toFixed(2)}%`, () => {
    // navigator.back();
    // }, () => {
    //   window.location.reload();
    // }, "其他相似K线", "再练一次");
  UIDisplayingHelper.showLoading();
  setTimeout(() => {
    UIDisplayingHelper.hideLoading();
    navigator.navigateToSimilarKlinePracticeResultView(this.props.match.params.stockCode,
      this.props.match.params.similarStockCode,
      this.props.location.state.similarKlineData,
      this.props.location.state.sourceKlineData,
      this.props.location.state.sourceStockName,
      this.state.currentYieldRate,
      this.actions);
  }, 500);
}
// View render
render() {
  const { historyData, myStockHistoryData } = this.state;

  return (

```

```

<ViewContainer style={{backgroundColor:"rgb(17,20,23,1.0)}}>

  <div className="kline-practice-view-container">

    { this.state.historyData &&
      <div className="header-panel">
        <div className="kline-practice-view-yield-info-container">
          <div className="yield-banner">
            <div className={`info-text ${this.state.currentYieldRate>=0?"positive":"negative"}`>>收益: {(this.state.currentYieldRate * 100).toFixed(2)}%</div>
            <div className="divided-line"/>
            <div className={`info-text ${this.state.empty?"empty":"full"}`>>{this.state.empty?"空仓":"持仓"}</div>
          </div>
        </div>
      </div>
    }

    { this.state.historyData &&
      <div className="chart-wrapper">
        <KlinePatternPracticeChart showMarkPoint showMarkLine
          myStockName={this.props.location.state.sourceStockName || ""}
          myStockHistoryData={myStockHistoryData}
          similarStockName={this.props.location.state.similarKlineData.name}
          historyData={historyData}
          ref={node => (this.klineChart = node)}>
        </div>
      </div>
    }
    {
      this.state.historyData &&
      <ActionButtonPanel>
        <ActionButtonPanel.Button type={ActionButtonPanel.Button.types.BUY}
          onClick={this.handleBuyButtonClick}
          disabled={!this.state.empty || this.state.tradeEnd}>
          买入
        </ActionButtonPanel.Button>
        <ActionButtonPanel.Button type={ActionButtonPanel.Button.types.SELL}
          onClick={this.handleSellButtonClick}
          disabled={this.state.empty || this.state.tradeEnd}>
          卖出
        </ActionButtonPanel.Button>
        <ActionButtonPanel.Button type={ActionButtonPanel.Button.types.GRAY}
          onClick={this.handleNoTradeButtonClick}
          disabled={this.state.tradeEnd}>
          观望
        </ActionButtonPanel.Button>
      </ActionButtonPanel>
    }
  </ViewContainer>
);
}
}
export default SimilarKlinePracticeView;

import React, { Component } from 'react';
import './PuzzleAnalysis.scss';

class PuzzleAnalysis extends Component {
  render() {
    return (
      <div className="pattern-practice-puzzle-analysis-wrapper">
        <div className="analysis-title-wrapper">
          <div className="analysis-div-line"></div>
        </div>
      </div>
    );
  }
}

```

```

    <div className="analysis-title">题目解析</div>
  </div>
  <div className="content">
    <div className="analysis">
      {this.props.children}
    </div>
    {
      this.props.showReview && this.props.details &&
      <div className="pattern-review">
        <div className="title">知识点回顾 [{this.props.details.cnName}] </div>
        <div className="desc">{this.props.details.desc}</div>
      </div>
    }
  </div>
</div>
);
}
}
export { PuzzleAnalysis };

import React, { Component } from 'react';
import PropTypes from 'prop-types';

class StockListCell extends Component {

  handleAddButtonClick = (e) => {
    // 阻止事件冒泡
    e.stopPropagation();
    e.nativeEvent.stopImmediatePropagation();
    if (!this.props.isFavor) {
      this.props.onAddButtonClick && this.props.onAddButtonClick();
    }
  }

  render() {
    return (
      <div className="cell-wrapper" onClick={ () => this.props.onClick && this.props.onClick() }>
        <div className="text-wrapper">
          <div className="title">{this.props.title}</div>
        </div>
        <div className="start-button-wrapper"
          style={ this.props.isFavor ? {color:"rgba(255,255,255,0.6)", fontSize:"0.2rem"} : {} }
          onClick={ this.handleAddButtonClick }>
          {this.props.isFavor?"已加入自选":"+"}
        </div>
      </div>
    );
  }
}

StockListCell.propTypes = {
  onAddButtonClick: PropTypes.func,
  onClick: PropTypes.func,
  title: PropTypes.string,
  subtitle: PropTypes.string,
  isFavor: PropTypes.bool
}

class StockList extends Component {
  render() {
    return (
      <div className="matching-stock-list-wrapper">
        {this.props.children}
      </div>
    );
  }
}

```

```

}

StockList.Cell = StockListCell;
export default StockList;

import React, { Component } from 'react';
import ViewContainer from '../../skeleton/ViewContainer';
import ActionButtonPanel from '../../component/ActionButtonPanel';
import UIDisplayingHelper from '../../component/helper/UIDisplayingHelper';
import KlinePanel from './KlinePanel';
import navigator from '../../routes/navigator';
import './SimilarKlinePracticeResultView.scss';

const wellDoneText = [
  "干得漂亮",
  "股市奇才",
  "很赞，继续加油",
  "你好，下一任世界首富",
  "你真的很会赚钱"
];

const justSoSoText = [
  "再接再厉吧，小韭菜",
  "我想你还可以做得更好",
  "下载一个操盘手再练练吧",
  "买卖点踩得不太准哦",
  "我想你应该回去再练一练"
];

class SimilarKlinePracticeResultView extends Component {

  state = {
    sourceKlineData: null,
    similarKlineData: null,
    yieldRate: 0.0,
    actions: []
  }

  componentDidMount() {
    this.setTitle("相似K线复盘练习结果");
    this.showNavBar(true);
    if (!this.props.location.state) {
      UIDisplayingHelper.showToast("无结果数据");
    } else {
      this.setState({
        sourceKlineData: this.props.location.state.sourceKlineData,
        similarKlineData: this.props.location.state.similarKlineData,
        yieldRate: this.props.location.state.yieldRate,
        actions: this.props.location.state.actions
      });
    }
  }

  handleButtonClick = () => {
    navigator.back();
  }

  // View render
  render() {
    return (
      <ViewContainer style={{backgroundColor:"rgba(17,20,23,1.0)", paddingBottom:"1.5rem"}}>

        {
          this.state.similarKlineData &&
          <div className="practice-result-panel">
            <div className="congratulation-text">

```

```

    {
      (() => {
        if (this.state.yieldRate > 0.05) {
          return wellDoneText[Math.floor(Math.random() * wellDoneText.length)];
        } else {
          return justSoSoText[Math.floor(Math.random() * justSoSoText.length)];
        }
      })()
    }
  </div>
  <div>
    className={`yield-rate ${this.state.yieldRate >= 0 ? "up" : "down"}`}>{(this.state.yieldRate
* 100).toFixed(2)}%
  </div>
</div>
}
{
  this.state.similarKlineData &&
  <KlinePanel similarDegree={this.state.similarKlineData.similarity}
    sourceStockName={this.props.location.state.sourceStockName || ""}
    sourceStockKlineData={this.state.sourceKlineData}
    similarStockName={({this.state.similarKlineData["name"] ||
this.state.similarKlineData["symbol"]})}
    similarStockKlineData={this.state.similarKlineData.dayKlineV0}
    actions={this.state.actions} />
}

<ActionButtonPanel>
  {
    this.state.similarKlineData
    ?
    <ActionButtonPanel.Button onClick={this.handleButtonClick}>
      复盘其他相似K线
    </ActionButtonPanel.Button>
    :
    <ActionButtonPanel.Button onClick={() => navigator.navigateToHomeView()}>
      返回首页
    </ActionButtonPanel.Button>
  }
</ActionButtonPanel>

</ViewContainer>
);
}
}

export default SimilarKlinePracticeResultView;

```